CHAPTER 21

Computational Geometry

21.0 Introduction

It is a safe bet that more computer cycles are expended on the formulas of computational geometry than on all other uses of computers put together. We include not just the computer's nominal CPU, of course, but also those other other, often vastly more powerful, CPUs hidden in the computer's graphics chipset, and in all the video entertainment and high-definition television boxes in the world.

Indeed, computational geometry, and the broader fields of computer graphics and computer vision in which it is embedded, have become central areas of computer science, supporting a huge industrial base of applied work and employment for computer scientists and program developers at all professional levels. It is impossible for us to do justice to this colossus in a single chapter. Yet, there are a number of elementary techniques from the field that ought to be in the repertory of any practicing computational scientist.

In this chapter we will build a body of methods sufficient to construct efficient Delaunay triangulations in two dimensions, and to use such triangulations for interpolating functions of two variables on an irregular grid, and other applications. In getting to this goal (and a bit beyond it) we will allow ourselves to be diverted into various other interesting, and often useful, topics, including:

- tree data structures for sets of points
- nearest-neighbor problems
- much about lines, triangles, and polygons
- spheres in n dimensions, and rotation matrices
- Voronoi and all that
- convex hulls
- minimum spanning trees
- finding intersecting objects

and more.

In the spirit of full disclosure, we must mention that our treatment of some of the most interesting topics in the above list will be restricted to the two-dimensional

case, even when the three-dimensional case may be equally relevant to computational science. The reason is simply one of space on the page. Three-dimensional algorithms are often more complex, have more special cases that must be treated, and generally result in codes that are too long for us to include. We have struggled to condense working, reasonably efficient, two-dimensional codes to an appropriate size for this chapter. You will be able to use these for two-dimensional problems, or you can mine them for understanding before seeking out three-dimensional solutions in the references.

An additional disclosure relates to our use of floating-point arithmetic, and our treatment of special cases of "exact" equality. Since floating-point numbers and their arithmetic are not exact, it usually does not make computational sense to test for cases of exact equality. However, historically, geometers have always distinguished, e.g., between a point being "inside" a triangle versus "on an edge" or "at a vertex." This has introduced a certain schizophrenia into the field. On the one hand (and especially before about 1990), practitioners have labored to create algorithms that use exact (integer) arithmetic, so that the traditional distinctions can be elegantly preserved. On the other hand (and especially after about 1990, when fast floating operations in special-purpose graphics processors started to be available), many of these niceties are no longer needed, and sloppiness at the level of "machine epsilon" can be tolerated in the interest of speed. In this chapter we are unapologetically in the sloppy camp. In boundary cases, our code is supposed to produce reasonable results, but not necessarily choose that specific reasonable result that you might think you want. Caveat emptor.

A less specific goal in this chapter is to give some of the "flavor" of the field of computational geometry. It is a flavor that deliciously combines elements of Euclid (pardon!) with elements of modern computer science and mathematics.

Some good general references are listed here.

CITED REFERENCES AND FURTHER READING:

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer). [Best-selling text, especially strong on references to the published literature.]

O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press). [Well written, with clear explanations and C code.]

Preparata, F.P. and Shamos, M.I. 1991, *Computational Geometry: An Introduction* (Berlin: Springer).

Schneider, P.J. and Eberly, D.H. 2003, *Geometric Tools for Computer Graphics* (San Francisco: Morgan Kaufmann). [Huge compendium of formulas and code.]

Bowyer, A. and Woodwark, J. 1983, *A Programmer's Geometry* (London: Butterworths). [Delightful classic, especially for those who get nostalgic at seeing Fortran printed in all uppercase.]

Glassner, A.S., ed. 1990, *Graphics Gems* (San Diego: Academic Press). [Series of books full of algorithmic tricks-of-the-trade.]

Arvo, J., ed. 1991, *Graphics Gems II* (San Diego: Academic Press).

Kirk, D., ed. 1992, *Graphics Gems III* (Cambridge, MA: Academic Press).

Heckbert, P.S., ed. 1994, *Graphics Gems IV* (Cambridge, MA: Academic Press).

Euclid, ca. 300BC, *Euclid's Elements*; reprinted 2002 (Santa Fe, NM: Green Lion Press).

21.1 Points and Boxes

A *point* $\mathbf{p}$ in a D-dimensional space is specified by its D Cartesian coordinates, $(x_0, x_1, \ldots, x_{D-1})$. Generally we will concern ourselves only with the cases $D = 2$ (points in a plane) and $D = 3$ (points in 3-space), but the concept is more general.

The representation in code follows just this paradigm. By eschewing special names for individual coordinates — like x, y, z — we keep the ability to loop easily over coordinates in D dimensions.

pointbox.h
```
template<Int DIM> struct Point {
Simple structure to represent a point in DIM dimensions.
    Doub x[DIM];                                          The coordinates.
    Point(const Point &p) {                               Copy constructor.
        for (Int i=0; i<DIM; i++) x[i] = p.x[i];
    }
    Point& operator= (const Point &p) {                   Assignment operator.
        for (Int i=0; i<DIM; i++) x[i] = p.x[i];
        return *this;
    }
    bool operator== (const Point &p) const {
        for (Int i=0; i<DIM; i++) if (x[i] != p.x[i]) return false;
        return true;
    }
    Point(Doub x0 = 0.0, Doub x1 = 0.0, Doub x2 = 0.0) {
        x[0] = x0;                                        Constructor by coordinate values. Arguments
        if (DIM > 1) x[1] = x1;                             beyond the required number are not used
        if (DIM > 2) x[2] = x2;                             and can be omitted.
        if (DIM > 3) throw("Point not implemented for DIM > 3");
    }
};
```

In the interest of concise code, the constructor above may pass some unnecessary default arguments of zero. You can easily clean this up if you care.

If we have two points $\mathbf{p}$ and $\mathbf{q}$, we can compute their distance d,

$$d = |\mathbf{p} - \mathbf{q}| = \left[\sum_{i=0}^{D-1} (p_i - q_i)^2 \right]^{1/2} \tag{21.1.1}$$

where p_i and q_i are now the respective Cartesian coordinates for each point.

In code, we have

pointbox.h
```
template<Int DIM> Doub dist(const Point<DIM> &p, const Point<DIM> &q) {
Returns the distance between two points in DIM dimensions.
    Doub dd = 0.0;
    for (Int j=0; j<DIM; j++) dd += SQR(q.x[j]-p.x[j]);
    return sqrt(dd);
}
```

Note that `dist` is not a member of the class `Point`, but rather a freestanding function whose arguments are `Points`. We will overload `dist` with other types of arguments, signifying other kinds of distances between objects.

21.1.1 Boxes

By a *box*, we mean a rectangle (for $D = 2$) or rectangular parallelepiped (for $D = 3$, a "brick" in other words) that is aligned with the coordinate axes. Boxes are

interesting because they can tessellate (that is, partition) D-dimensional space, and they can contain other objects. Indeed, every finite, extended object has a *bounding box*, which is the unique smallest box that contains it. One way to represent a box is by the points at two special, diagonally opposite, corners. The first point ("low") has coordinate values that are the minima on the surface of the box; the second point ("high") has coordinate values that are the maxima. All other corners of a box, it should be obvious, have coordinate values that are, dimension by dimension, either the value of "low" or the value of "high"; and all such permutations are corners, 2^D in all.

The code follows this description:

pointbox.h
```
template<Int DIM> struct Box {
```
Structure to represent a Cartesian box in DIM dimensions.
```
    Point<DIM> lo, hi;
    Box() {}
    Box(const Point<DIM> &mylo, const Point<DIM> &myhi) : lo(mylo), hi(myhi) {}
};
```
Diagonally opposite corners (min of all coordinates and max of all coordinates) are stored as two points.

Note that a copy constructor and assignment operator are not needed, since by default the two `Points` will be appropriately copied or assigned (one convenience of this representation).

A point can be either outside a box, inside it, or — in principle — on its surface. As mentioned in §21.0, we represent all coordinates as (approximate) floating-point numbers, not (exact) integers, so it would not be prudent to depend on any exact equalities of coordinate values or distances. We will be careful, therefore, not to put too much credence in the idea of the exact surface of a box; usually we'll consider the surface (should some exact equality happen to hold) as a part of the box's interior.

If a point is outside a box, then we define its distance from the box to be the distance to the nearest point on the surface of (or inside) the box. A glance at Figure 21.1.1 shows that this distance is the Pythagorean sum (that is, square root of sum of squares) of the distances from the point to some — but not all — of the hyperplanes that bound the box. The rule is that when a point has a coordinate that is greater than the corresponding max of the box, or less than the corresponding min, then *that* coordinate contributes to the sum. When the point has a coordinate between the max and min, then it does *not* contribute to the sum, since (along that coordinate) the shortest line can be perpendicular to the hyperplane. When a point is inside, or on the surface of, a box, we define its distance to the box to be zero.

These definitions of distance are embodied in the following code.

pointbox.h
```
template<Int DIM> Doub dist(const Box<DIM> &b, const Point<DIM> &p) {
```
If point p lies outside box b, the distance to the nearest point on b is returned. If p is inside b or on its surface, zero is returned.
```
    Doub dd = 0;
    for (Int i=0; i<DIM; i++) {
        if (p.x[i]<b.lo.x[i]) dd += SQR(p.x[i]-b.lo.x[i]);
        if (p.x[i]>b.hi.x[i]) dd += SQR(p.x[i]-b.hi.x[i]);
    }
    return sqrt(dd);
}
```

Frequently we want to know if a point is inside or outside a box. The above `dist` routine can be used for this. A positive return means outside, zero means

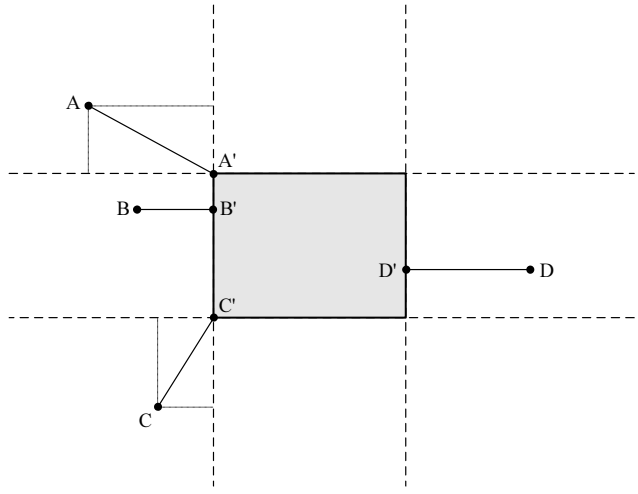


Figure 21.1.1. Distance from a point to a D-dimensional box. The general formula (as for lines AA' and CC') is a Pythagorean sum of D distances to the plane that includes the nearer side of the box. But when the point is between two such parallel planes (as for BB' and DD') then the corresponding coordinate is omitted from the sum.

inside. If inside-versus-outside, and not distance, is *all* you want to know, then some streamlining is possible: Replace `dd` by a Boolean variable, substitute logical-or's for the additions, and of course omit the square root. The logic remains otherwise the same.

21.1.2 Nodes for Binary Trees of Boxes

In the next section we will construct a *binary tree* of nested boxes, wherein each box is subdivided into, and linked to, two daughter boxes. Each box in the tree will also contain a list of points that lie inside the box. For use in such a task, we here give a structure, derived from the `Box` structure, but with additional variables that can point to a mother box, two daughter boxes, and the low and high indices on a list of points (designating the range of points inside the box). The constructor just sets all the values explicitly.

kdtree.h

```
template<Int DIM> struct Boxnode : Box<DIM> {
Node in a binary tree of boxes containing points. See text for details.
    Int mom, dau1, dau2, ptlo, pthi;
    Boxnode() {}
    Boxnode(Point<DIM> mylo, Point<DIM> myhi, Int mymom, Int myd1,
        Int myd2, Int myptlo, Int mypthi) :
        Box<DIM>(mylo, myhi), mom(mymom), dau1(myd1), dau2(myd2),
        ptlo(myptlo), pthi(mypthi) {}
};
```

21.2 KD Trees and Nearest-Neighbor Finding

Once, long ago, the term "kd tree" (or "k-D tree") was an abbreviation for "k-dimensional tree." However, the term has come to mean a very specific, and

useful, kind of tree structure for partitioning points, especially in small numbers of dimensions, like 2 or 3. A KD tree that contains N points can be constructed in $O(N \log N)$ time and $O(N)$ space. Once constructed, the KD tree facilitates such operations as finding a point's nearest neighbor in $O(\log N)$ time, or all nearest neighbors in $O(N \log N)$ time. KD trees were first described by Bentley [8] in 1975. Let's see how this works.

Start with a very large box, one that easily contains all possible points that are of interest. There is no penalty for making this *root box* humongous, so coordinates of $\pm 10^{99}$ are fine. Now generate a list of N points (of interest to your application) that lie inside the root box. The defining principles of a KD tree are

- Boxes are successively partitioned into two daughter boxes.
- Each partition is along the axis of one coordinate.
- The coordinates are used cyclically in successive partitions.
- In making the partition, the position of the "cut" is chosen to leave equal numbers of points on the two sides (or differing by one in the case of an odd number of points).

Within these principles, there are some arbitrary design choices to be made. In the implementation below, the partition "cut" goes exactly through one of the points (i.e., shares one of its coordinate values). This avoids a bit of extra bookkeeping incurred by other possible choices. Also, we terminate the tree when a box node contains either one or two points, avoiding the additional partitioning of two-point boxes into two one-point boxes. This choice is natural because the `Boxnode` structure already has pointers to two points (`ptlo` and `pthi`), and it reduces the total number of stored boxes by as much as 50%.

With these principles and design rules in mind, you can decode Figure 21.2.1, which shows a two-dimensional KD tree containing 1000 points. (As a bit of artistic license, the root box in the figure has been shrunk to just contain the points, instead of being off near infinity.)

Interestingly, given N, the number of points, it is possible to give an exact formula for the number of boxes generated by our KD tree partition rules. (This makes memory allocation for the tree very straightforward.) If $N_B(N)$ is the number of boxes needed for N points, then two obvious recurrence relations describe what happens in the initial partitioning of $2n$ points into n plus n, or $2n-1$ points into n plus $n-1$:

$$\begin{aligned} N_B(2n) &= 2N_B(n) + 1 \\ N_B(2n-1) &= N_B(n) + N_B(n-1) + 1 \end{aligned} \tag{21.2.1}$$

The $+1$ in both formulas refers to the additional mother box that "glues together" two daughter partial trees at each stage. The solution to these recurrences is

$$N_B(N) = \min(M-1,\ 2N - \tfrac{1}{2}M - 1) \tag{21.2.2}$$

where M is the smallest power of 2 greater or equal to N, that is,

$$M = 2^{\lceil \log_2 N \rceil} \tag{21.2.3}$$

(You can verify this solution by induction, working out the various possibilities of the min function. Or — much more fun — you can write a program to verify it numerically for $N < 10^9$, say.)

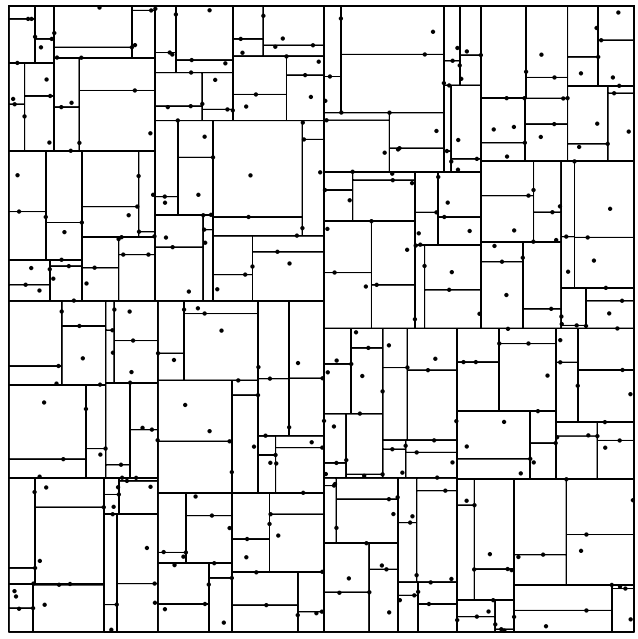


Figure 21.2.1. KD tree constructed from 1000 points in the plane. The first subdivision is visible as a full-height vertical line about halfway across the figure. The next subdivisions are horizontal lines, extending halfway across the figure. The subdivisions alternate between horizontal and vertical, and partition into (nearly) equal numbers of points at each stage. This tree terminates when there are either one or two points in a box (one of which is usually on the box boundary).

21.2.1 Implementation of the KD Tree

We implement the KD tree as a structure that gets built from a vector of `Points` and contains methods that embody the principal applications that we will discuss below, mainly various kinds of nearest-neighbor problems.

kdtree.h

```
template<Int DIM> struct KDtree {
```
Structure for implementing a KD tree.
```
    static const Doub BIG;                  Size of the root box, value set below.
    Int nboxes, npts;                       Number of boxes, number of points.
    vector< Point<DIM> > &ptss;             Reference to the vector of points in the KD tree.
    Boxnode<DIM> *boxes;                    The array of Boxnodes that form the tree.
    VecInt ptindx, rptindx;                 Index of points (see text), and reverse index.
    Doub *coords;                           Point coordinates rearranged contiguously.
    KDtree(vector< Point<DIM> > &pts);      Constructor.
    ~KDtree() {delete [] boxes;}
```
Next, utility functions for use after the tree is constructed. See below.
```
    Doub disti(Int jpt, Int kpt);
    Int locate(Point<DIM> pt);
    Int locate(Int jpt);
```
Next, applications that use the KD tree. See text.
```
    Int nearest(Int jpt);
    Int nearest(Point<DIM> pt);
    void nnearest(Int jpt, Int *nn, Doub *dn, Int n);
    static void sift_down(Doub *heap, Int *ndx, Int nn);    Used by nnearest.
    Int locatenear(Point<DIM> pt, Doub r, Int *list, Int nmax);
};

template<Int DIM> const Doub KDtree<DIM>::BIG(1.0e99);
```

Note that the KDtree structure keeps a reference to the vector of Points that created it. This is used in some of the applications and has the implication that the user should not modify the vector of points while its derived KD tree is in scope. The array coords is an internal representation of the points-vector that is used during the construction of the KD tree, and then is immediately returned to the memory pool.

What makes the KD tree fast to construct is the existence of fast partition algorithms, $O(N)$ in time, that not only find the median value in an array of N values, but also move all smaller values to one side of the array and all larger values to the other. We already met such an algorithm in §8.5, in the routine select. Here, we need a slight variant, selecti, that partitions an array of integers not by their values, but by using them to index a separate array of values that remain unaltered. Because we will want to partition subsegments of arrays, we pass all references to the arrays by address.

kdtree.h

```
Int selecti(const Int k, Int *indx, Int n, Doub *arr)
```

Permutes indx[0..n-1] to make arr[indx[0..k-1]] ≤ arr[indx[k]] ≤ arr[indx[k+1..n-1]]. The array arr is not modified. See comments in the routine select.

```
{
    Int i,ia,ir,j,l,mid;
    Doub a;

    l=0;
    ir=n-1;
    for (;;) {
        if (ir <= l+1) {
            if (ir == l+1 && arr[indx[ir]] < arr[indx[l]])
                SWAP(indx[l],indx[ir]);
            return indx[k];
        } else {
            mid=(l+ir) >> 1;
            SWAP(indx[mid],indx[l+1]);
            if (arr[indx[l]] > arr[indx[ir]]) SWAP(indx[l],indx[ir]);
            if (arr[indx[l+1]] > arr[indx[ir]]) SWAP(indx[l+1],indx[ir]);
            if (arr[indx[l]] > arr[indx[l+1]]) SWAP(indx[l],indx[l+1]);
            i=l+1;
            j=ir;
            ia = indx[l+1];
            a=arr[ia];
            for (;;) {
                do i++; while (arr[indx[i]] < a);
                do j--; while (arr[indx[j]] > a);
                if (j < i) break;
                SWAP(indx[i],indx[j]);
            }
            indx[l+1]=indx[j];
            indx[j]=ia;
            if (j >= k) ir=j-1;
            if (j <= k) l=i;
        }
    }
}
```

The basic strategy for constructing the KD tree is this: Set up an array of integers that index the N points (ptindx, below). Next, copy all the point coordinates into an array (coords) in which all the x_0 coordinates are contiguous, followed by all the x_1 coordinates, and so on through all the dimensions. Now use selecti to partition (and rearrange) the index of points according to the value of their x_0 co-

ordinates, with half the points on each side of the partition. These two halves, now viewed as separate arrays, contain the points in two new daughter boxes. Now partition each of them into half by the value of the x_1 coordinate. And so on, recursively, going through the coordinates cyclically.

The recursion is so simple that it is easy to code it as a simple "pending task list," thus avoiding the overhead of recursive function calls. A pending task consists of an index pointing to the box ready for further partitioning (the expectant mother, as it were) and a value that remembers which dimension is next to partition along. Because the tree is constructed "depth first," the task list never grows larger than the log of the total number of boxes. Every new daughter box is born with a pointer to its mother, and pointers to its beginning and end elements in the points index array `ptindx`. Although these elements will generally be permuted in subsequent partitionings, none will ever be moved out of the range specified when a daughter box is first created. That is why the whole process can be done in a single point-index array, with all boxes simply pointing into some subrange of that array.

The `KDtree` constructor, below, should now be straightforward to understand.

kdtree.h

```
template<Int DIM> KDtree<DIM>::KDtree(vector< Point<DIM> > &pts) :
ptss(pts), npts(pts.size()), ptindx(npts), rptindx(npts) {
Construct a KD tree from a vector of points.
    Int ntmp,m,k,kk,j,nowtask,jbox,np,tmom,tdim,ptlo,pthi;
    Int *hp;
    Doub *cp;
    Int taskmom[50], taskdim[50];                Enough stack for 2^50 points!
    for (k=0; k<npts; k++) ptindx[k] = k;        Initialize the index of points.
    Calculate the number of boxes and allocate memory for them.
    m = 1;
    for (ntmp = npts; ntmp; ntmp >>= 1) {
        m <<= 1;
    }
    nboxes = 2*npts - (m >> 1);
    if (m < nboxes) nboxes = m;
    nboxes--;
    boxes = new Boxnode<DIM>[nboxes];
    Copy the point coordinates into a contiguous array.
    coords = new Doub[DIM*npts];
    for (j=0, kk=0; j<DIM; j++, kk += npts) {
        for (k=0; k<npts; k++) coords[kk+k] = pts[k].x[j];
    }
    Initialize the root box and put it on the task list for subdivision.
    Point<DIM> lo(-BIG,-BIG,-BIG), hi(BIG,BIG,BIG);      Syntax OK for 2-D too.
    boxes[0] = Boxnode<DIM>(lo, hi, 0, 0, 0, 0, npts-1);
    jbox = 0;
    taskmom[1] = 0;                              Which box.
    taskdim[1] = 0;                              Which dimension.
    nowtask = 1;
    while (nowtask) {                            Main loop over pending tasks.
        tmom = taskmom[nowtask];
        tdim = taskdim[nowtask--];
        ptlo = boxes[tmom].ptlo;
        pthi = boxes[tmom].pthi;
        hp = &ptindx[ptlo];                      Points to left end of subdivision.
        cp = &coords[tdim*npts];                 Points to coordinate list for current dim.
        np = pthi - ptlo + 1;                    Number of points in the subdivision.
        kk = (np-1)/2;                           Index of last point on left (boundary point).
        (void) selecti(kk,hp,np,cp);             Here is where all the work is done.
        Now create the daughters and push them onto the task list if they need further subdividing.
```

```
            hi = boxes[tmom].hi;
            lo = boxes[tmom].lo;
            hi.x[tdim] = lo.x[tdim] = coords[tdim*npts + hp[kk]];
            boxes[++jbox] = Boxnode<DIM>(boxes[tmom].lo,hi,tmom,0,0,ptlo,ptlo+kk);
            boxes[++jbox] = Boxnode<DIM>(lo,boxes[tmom].hi,tmom,0,0,ptlo+kk+1,pthi);
            boxes[tmom].dau1 = jbox-1;
            boxes[tmom].dau2 = jbox;
            if (kk > 1) {
                taskmom[++nowtask] = jbox-1;
                taskdim[nowtask] = (tdim+1) % DIM;
            }
            if (np - kk > 3) {
                taskmom[++nowtask] = jbox;
                taskdim[nowtask] = (tdim+1) % DIM;
            }
        }
        for (j=0; j<npts; j++) rptindx[ptindx[j]] = j;      Create reverse index.
        delete [] coords;                                   Don't need them anymore.
    }
```

There are a small number of utility functions that are easy to provide. Although we generally prefer to have our distance (`dist`) functions be freestanding, it is useful to have a `KDtree` member routine that returns the distance between two points in a KD tree, referenced by their integer position in the underlying vector of points.

kdtree.h

```
template<Int DIM> Doub KDtree<DIM>::disti(Int jpt, Int kpt) {
```

Returns the distance between two points in the `kdtree` given their indices in the array of points, but returns a large value if the points are identical.

```
    if (jpt == kpt) return BIG;
    else return dist(ptss[jpt], ptss[kpt]);
}
```

There is a special reason for returning `BIG` when the two points are identical: Later, when we find a point's nearest neighbor, we don't want the invariable answer to be "itself!"

Another simple function takes an arbitrary `Point` as the argument and returns the index of the box that uniquely contains it. In this function we first see an example of traversing the tree hierarchically, starting at the root box and then choosing only one of two daughter boxes at each stage. Also, by keeping track of which dimension is next to be partitioned on (`jdim`, below), we need only check one of the point's coordinates at each stage. Evidently, the whole process is $O(\log N)$ in time, since there can be only that many levels in the tree.

kdtree.h

```
template<Int DIM> Int KDtree<DIM>::locate(Point<DIM> pt) {
```

Given an arbitrary point `pt`, return the index of which `kdtree` box it is in.

```
    Int nb,d1,jdim;
    nb = jdim = 0;                          Start with the root box.
    while (boxes[nb].dau1) {                As far as possible down the tree.
        d1 = boxes[nb].dau1;
        if (pt.x[jdim] <= boxes[d1].hi.x[jdim]) nb=d1;
        else nb=boxes[nb].dau2;
        jdim = ++jdim % DIM;                Increment the dimension cyclically.
    }
    return nb;
}
```

The actual `Box` can be obtained from the returned integer, say `j`, by referencing `boxes[j]` in the `KDtree`, since `Boxnode` is a derived class of `Box`.

A very similar utility returns the index of the box that contains one of the points used to construct the KDtree. This is not necessarily the same box as the above routine would return, because of the possibility of multiple ties in coordinate values, in which case some tied points can lie on one side of the median partition and others on the other side.

kdtree.h

```
template<Int DIM> Int KDtree<DIM>::locate(Int jpt) {
```

Given the index of a point in the kdtree, return the index of which box it is in.

```
    Int nb,d1,jh;
    jh = rptindx[jpt];                    The reverse index tells where the point lies in the
    nb = 0;                                   index of points.
    while (boxes[nb].dau1) {
        d1 = boxes[nb].dau1;
        if (jh <= boxes[d1].pthi) nb=d1;
        else nb = boxes[nb].dau2;
    }
    return nb;
}
```

21.2.2 Applications of KD Trees

Most applications of KD trees make use of locality properties of its nested boxes. This is best seen in a few examples.

Suppose we want to know which of the N points in a KD tree is closest to an arbitrary point $\mathbf{p}$ (not necessarily one of the points in the tree). Without the tree, this is evidently a calculation that requires $O(N)$ operations, as we compare $\mathbf{p}$ to each candidate point in turn. However, if we have invested the $O(N \log N)$ operations required to construct the tree, then we can proceed in the following way. First, find the box in which $\mathbf{p}$ lies, and find the closest point in the tree that lies in that box. This takes $O(\log N)$ operations, as we saw above. The found point might in fact be the nearest neighbor (we don't know yet), but in any case its distance is now an upper bound on how far away the true nearest neighbor can be.

Second, traverse the tree by a depth-first recursion (exactly the way we did when we constructed the tree). As we encounter each new box, we check whether it could possibly contain a point closer than the nearest point found so far. Since we start with a point that is already pretty close (in the same box as $\mathbf{p}$), most boxes get rejected at this step. When a box is rejected, we *don't* need to open its daughter boxes, so a whole branch of the tree gets "pruned." On average, only about $O(\log N)$ boxes actually get opened, so the total work load to find the nearest point is $O(\log N)$.

If we are really interested in only a single point $\mathbf{p}$, then the "slow," $O(N)$, method would have been faster. But if we are repeating the operation for many different points $\mathbf{p}_i$, comparing to the same N points in the tree each time, then calling the following routine for each $\mathbf{p}_i$ in turn is a big win.

kdtree.h

```
template<Int DIM> Int KDtree<DIM>::nearest(Point<DIM> pt) {
```

Given an arbitrary location pt, return the index of the nearest point in the kdtree.

```
    Int i,k,nrst,ntask;
    Int task[50];                         Stack for boxes waiting to be opened.
    Doub dnrst = BIG, d;
```

First stage, we find the nearest kdtree point in same box as pt.

```
    k = locate(pt);                       Which box is pt in?
    for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {   Find nearest.
        d = dist(ptss[ptindx[i]],pt);
```

```
            if (d < dnrst) {
                nrst = ptindx[i];
                dnrst = d;
            }
        }
        Second stage, we traverse the tree opening only possibly better boxes.
        task[1] = 0;
        ntask = 1;
        while (ntask) {
            k = task[ntask--];
            if (dist(boxes[k],pt) < dnrst) {          Distance to closest point in box.
                if (boxes[k].dau1) {                  If not an end node, put on task list.
                    task[++ntask] = boxes[k].dau1;
                    task[++ntask] = boxes[k].dau2;
                } else {                              Check the 1 or 2 points in the box.
                    for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                        d = dist(ptss[ptindx[i]],pt);
                        if (d < dnrst) {
                            nrst = ptindx[i];
                            dnrst = d;
                        }
                    }
                }
            }
        }
        return nrst;
    }
```

What if we want to know the nearest-neighbor point not of an arbitrary location, but of one of the points stored in the KD tree? The above routine won't do. If we send it a point in the tree, it will give the obvious result that the point is its own nearest neighbor! We need to modify the routine so as to use `disti` from `KDtree`, which defined a point's self-distance as being large, rather than small.

An additional useful feature is to find not the single nearest neighbor, but the n nearest neighbors for some specified $n < N - 1$. The trick here is to avoid making the algorithm $O(n \log N)$, which is what it would be if, for each candidate point, we compared the candidate to all n of the best points so far. A good way to proceed is with a heap structure, as described in §8.3 and used (for a very similar purpose) in the routine `hpsel` in §8.5. The work load then scales as $O(\log n \log N)$.

The following routine is coded so as to lose hardly any efficiency in the case $n = 1$ (find the single nearest neighbor) while using a heap structure in the case $n > 1$.

kdtree.h

```
    template<Int DIM> void KDtree<DIM>::nnearest(Int jpt, Int *nn, Doub *dn, Int n)
    Given the index jpt of a point in a kdtree, return a list nn[0..n-1] of the indices of the n
    points in the tree nearest to point j, and a list dd[0..n-1] of their distances.
    {
        Int i,k,ntask,kp;
        Int task[50];                              Stack for boxes to be opened.
        Doub d;
        if (n > npts-1) throw("too many neighbors requested");
        for (i=0; i<n; i++) dn[i] = BIG;
        Find smallest mother box with enough points to initialize the heap.
        kp = boxes[locate(jpt)].mom;
        while (boxes[kp].pthi - boxes[kp].ptlo < n) kp = boxes[kp].mom;
        Examine its points and save the n closest.
        for (i=boxes[kp].ptlo; i<=boxes[kp].pthi; i++) {
            if (jpt == ptindx[i]) continue;
            d = disti(ptindx[i],jpt);
```

```
            if (d < dn[0]) {
                dn[0] = d;
                nn[0] = ptindx[i];
                if (n>1) sift_down(dn,nn,n);    Maintain the heap structure.
            }
        }
        Now we traverse the tree opening only possibly better boxes.
        task[1] = 0;
        ntask = 1;
        while (ntask) {
            k = task[ntask--];
            if (k == kp) continue;              Don't redo the box used to initialize.
            if (dist(boxes[k],ptss[jpt]) < dn[0]) {
                if (boxes[k].dau1) {            If not an end node, put on task list.
                    task[++ntask] = boxes[k].dau1;
                    task[++ntask] = boxes[k].dau2;
                } else {                        Check the 1 or 2 points in the box.
                    for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                        d = disti(ptindx[i],jpt);
                        if (d < dn[0]) {
                            dn[0] = d;
                            nn[0] = ptindx[i];
                            if (n>1) sift_down(dn,nn,n);  Maintain the heap.
                        }
                    }
                }
            }
        }
        return;
    }
```

The following routine is used by the above for the sift-down process on the heap, differing from the `sift_down` used by `hpsort` (§8.3) only in its tailored interface for the present application, and the fact that it simultaneously rearranges two arrays, the distances (forming a heap) and the corresponding point numbers.

kdtree.h

```
template<Int DIM> void KDtree<DIM>::sift_down(Doub *heap, Int *ndx, Int nn) {
    Fix heap[0..nn-1], whose first element (only) may be wrongly filed. Make a corresponding
    permutation in ndx[0..nn-1]. The algorithm is identical to that used by sift_down in hpsort.
        Int n = nn - 1;
        Int j,jold,ia;
        Doub a;
        a = heap[0];
        ia = ndx[0];
        jold = 0;
        j = 1;
        while (j <= n) {
            if (j < n && heap[j] < heap[j+1]) j++;
            if (a >= heap[j]) break;
            heap[jold] = heap[j];
            ndx[jold] = ndx[j];
            jold = j;
            j = 2*j + 1;
        }
        heap[jold] = a;
        ndx[jold] = ia;
    }
```

As a final illustrative example, here is how to find all points in a KD tree that lie within a specified radius r of some arbitrary location $\mathbf{p}$.

kdtree.h

```
template<Int DIM>
Int KDtree<DIM>::locatenear(Point<DIM> pt, Doub r, Int *list, Int nmax) {
```

Given a point pt and radius r, returns a value nret such that list[0..nret-1] is a list of all kdtree points within a radius r of pt, up to a user-specified maximum of nmax points.

```
    Int k,i,nb,nbold,nret,ntask,jdim,d1,d2;
    Int task[50];
    nb = jdim = nret = 0;
    if (r < 0.0) throw("radius must be nonnegative");
    Find the smallest box that contains the "ball" of radius r.
    while (boxes[nb].dau1) {
        nbold = nb;
        d1 = boxes[nb].dau1;
        d2 = boxes[nb].dau2;
        Only need to check the dimension that divides the daughters.
        if (pt.x[jdim] + r <= boxes[d1].hi.x[jdim]) nb = d1;
        else if (pt.x[jdim] - r >= boxes[d2].lo.x[jdim]) nb = d2;
        jdim = ++jdim % DIM;
        if (nb == nbold) break;                          Neither daughter encloses the ball.
    }
    Now traverse the tree below the starting box only as needed.
    task[1] = nb;
    ntask = 1;
    while (ntask) {
        k = task[ntask--];
        if (dist(boxes[k],pt) > r) continue;             Box and ball are disjoint.
        if (boxes[k].dau1) {                             Expand box further when possible.
            task[++ntask] = boxes[k].dau1;
            task[++ntask] = boxes[k].dau2;
        } else {                                         Otherwise process points in the box.
            for (i=boxes[k].ptlo; i<=boxes[k].pthi; i++) {
                if (dist(ptss[ptindx[i]],pt) <= r && nret < nmax)
                    list[nret++] = ptindx[i];
                if (nret == nmax) return nmax;           Not enough space!
            }
        }
    }
    return nret;
}
```

You might wonder why the above routine doesn't also use the tree structure to find cases where a box lies entirely inside the "ball" of radius r, in which case it could add the box's points to the output list without further opening of its daughters. The improvement is potentially a factor of $O(\log n)$, where n is the typical number of neighbors returned. The resulting routine is slightly too long for us to include, however. A good exercise is to code this modification yourself. You'll see that it is harder to check whether a box is inside a ball than vice versa: You have to check all 2^D corners of the box, not just the diagonally opposite "low" and "high" ones.

CITED REFERENCES AND FURTHER READING:

Bentley, J.L. 1975, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM*, vol. 18, pp. 509–517.

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), §5.2.

Samet, H. 1990, *The Design and Analysis of Spatial Data Structures* (Reading, MA: Addison-Wesley).

21.3 Triangles in Two and Three Dimensions

Not since the time of Euclid has the lowly triangle attracted as much attention as it does today in computer graphics. Triangles and *triangulation* (the decomposition, or approximation, of complicated geometrical objects using only triangles) are at the heart of practically every computer-generated image.

Three points, call them **a**, **b**, **c**, define a triangle. They are its *vertices*. If the points are two-dimensional, the triangle lies in the two-dimensional plane. If the points have higher dimensionality, then the triangle floats in the corresponding D-dimensional space (most commonly $D = 3$). For now, consider only the former case, with $D = 2$, so that **a** has coordinates (a_0, a_1), and similarly for **b** and **c**.

Area. The area $\mathcal{A}(\mathbf{abc})$ of the triangle $\triangle\mathbf{abc}$ can be written in a number of equivalent ways, including

$$\begin{aligned} 2\mathcal{A}(\mathbf{abc}) &= \begin{vmatrix} a_0 & a_1 & 1 \\ b_0 & b_1 & 1 \\ c_0 & c_1 & 1 \end{vmatrix} \\ &= (\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a}) = (b_0-a_0)(c_1-a_1) - (b_1-a_1)(c_0-a_0) \\ &= (\mathbf{c}-\mathbf{b}) \times (\mathbf{a}-\mathbf{b}) = (c_0-b_0)(a_1-b_1) - (c_1-b_1)(a_0-b_0) \\ &= (\mathbf{a}-\mathbf{c}) \times (\mathbf{b}-\mathbf{c}) = (a_0-c_0)(b_1-c_1) - (a_1-c_1)(b_0-c_0) \end{aligned} \tag{21.3.1}$$

Here $\times$ denotes the vector cross product, defined in two dimensions simply by

$$\mathbf{A} \times \mathbf{B} = A_0 B_1 - B_1 A_0 \qquad \text{(two dimensions only)} \tag{21.3.2}$$

Below, when we consider triangles in three dimensions, it will be the vector cross product forms in equation (21.3.1) that give a generalized formula for the area. Let us also note in passing that the formulas for area are separately *linear* in each of the six coordinates a_0, a_1, b_0, b_1, c_0, and c_1.

Equation (21.3.1) can yield a value that is positive, zero, or negative: The area is a *signed area*. By convention (embodied in equation 21.3.1), the area is positive if a traversal from **a** to **b** to **c** goes *counterclockwise (CCW)* around the triangle, and negative if it goes *clockwise (CW)*. The area is zero if and only if the three points are collinear, in which case the triangle is degenerate. (In the formulas that follow, we will generally assume the nondegenerate case.)

The absolute value $|\mathcal{A}|$ is the (unsigned) "area" of the triangle in the conventional geometrical sense. It can also be calculated directly from the side lengths d_{ab}, d_{bc}, and d_{ca} as follows:

$$|\mathcal{A}| = \sqrt{s(s-d_{ab})(s-d_{bc})(s-d_{ca})} \tag{21.3.3}$$

where s is half the perimeter,

$$s \equiv \tfrac{1}{2}(d_{ab} + d_{bc} + d_{ca}) \tag{21.3.4}$$

(Does it go without saying that you compute the side lengths by taking the coordinate differences and using the Pythagorean theorem?)

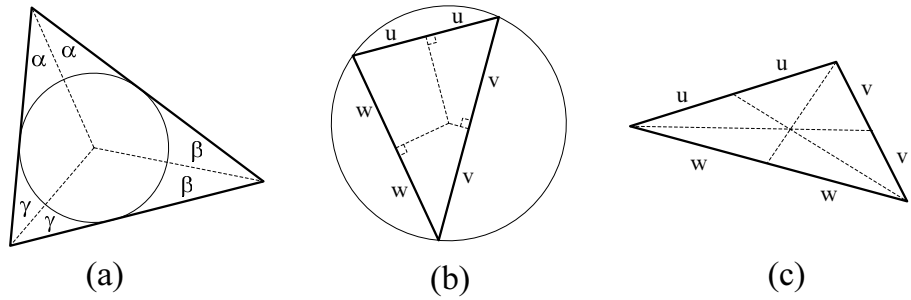


Figure 21.3.1. Three kinds of triangle centers. (a) Incircle and incenter; bisectors of the three vertex angles meet at the incenter. (b) Circumcircle and circumcenter; perpendicular bisectors of the edges meet at the circumcenter. (c) Centroid; lines from the edge midpoints to the opposite vertices meet at the centroid.

Related Circles. Every nondegenerate triangle has an *inscribed circle* or *incircle*, which is the largest circle that can be drawn inside the triangle. The incircle is tangent to all three sides of the triangle. Lines from its center, the *incenter*, to each vertex bisect the angle at that vertex (see Figure 21.3.1). If $\mathbf{q}$ is the incenter point, with coordinates (q_0, q_1), then its location is given by

$$q_i = \frac{1}{2s}(d_{bc}a_i + d_{ca}b_i + d_{ab}c_i) \qquad (i = 0, 1) \tag{21.3.5}$$

while its radius is given by

$$r_{\text{in}} = \left(\frac{(s - d_{ab})(s - d_{bc})(s - d_{ca})}{s} \right)^{1/2} \tag{21.3.6}$$

Every nondegenerate triangle also has a *circumscribed circle* or *circumcircle*, which is the unique circle that goes through its three vertices. Suppose $\mathbf{Q}$ is the *circumcenter* point, with coordinates (Q_0, Q_1). Let $[ba]_0$ and $[ba]_1$ denote the coordinate differences $b_0 - a_0$ and $b_1 - a_1$, respectively; and similarly for $[ca]_0$ and $[ca]_1$. Then, in 2×2 determinant form,

$$\begin{aligned} Q_0 &= a_0 + \frac{1}{2} \begin{vmatrix} ([ba]_0)^2 + ([ba]_1)^2 & [ba]_1 \\ ([ca]_0)^2 + ([ca]_1)^2 & [ca]_1 \end{vmatrix} \Bigg/ \begin{vmatrix} [ba]_0 & [ba]_1 \\ [ca]_0 & [ca]_1 \end{vmatrix} \\ Q_1 &= a_1 + \frac{1}{2} \begin{vmatrix} [ba]_0 & ([ba]_0)^2 + ([ba]_1)^2 \\ [ca]_0 & ([ca]_0)^2 + ([ca]_1)^2 \end{vmatrix} \Bigg/ \begin{vmatrix} [ba]_0 & [ba]_1 \\ [ca]_0 & [ca]_1 \end{vmatrix} \end{aligned} \tag{21.3.7}$$

The circumcenter is, by definition, the same distance from all three vertices. Therefore the radius of the circumcircle is

$$r_{\text{circum}} = \sqrt{(Q_0 - a_0)^2 + (Q_1 - a_1)^2} \tag{21.3.8}$$

where Q_0 and Q_1 are given above. (Obviously you can save the semi-final results in equation 21.3.7 for this computation, before adding a_0 or a_1.)

Later, in §21.6, we will be calculating a lot of circumcircles. We use the following simple definition of a structure `Circle`, and a routine `circumcircle()` that directly implements equations (21.3.7) and (21.3.8).

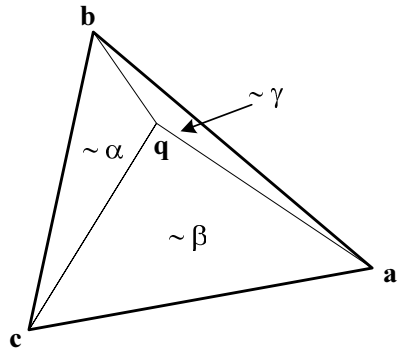


Figure 21.3.2. Any point in the plane **q** can be expressed as a linear combination of a triangle's three vertices. The coefficients (α, β, γ), called barycentric coordinates, sum to 1 and are proportional to the areas of $\triangle\mathbf{qbc}$, $\triangle\mathbf{qca}$, and $\triangle\mathbf{qab}$, respectively.

circumcircle.h

```
struct Circle {
    Point<2> center;
    Doub radius;
    Circle(const Point<2> &cen, Doub rad) : center(cen), radius(rad) {}
};

Circle circumcircle(Point<2> a, Point<2> b, Point<2> c) {
    Doub a0,a1,c0,c1,det,asq,csq,ctr0,ctr1,rad2;
    a0 = a.x[0] - b.x[0]; a1 = a.x[1] - b.x[1];
    c0 = c.x[0] - b.x[0]; c1 = c.x[1] - b.x[1];
    det = a0*c1 - c0*a1;
    if (det == 0.0) throw("no circle thru colinear points");
    det = 0.5/det;
    asq = a0*a0 + a1*a1;
    csq = c0*c0 + c1*c1;
    ctr0 = det*(asq*c1 - csq*a1);
    ctr1 = det*(csq*a0 - asq*c0);
    rad2 = ctr0*ctr0 + ctr1*ctr1;
    return Circle(Point<2>(ctr0 + b.x[0], ctr1 + b.x[1]), sqrt(rad2));
}
```

Centroid and Barycentric Coordinates. Distinct from both its incenter and its circumcenter is a triangle's centroid, or center of gravity, **M**. This point lies at the intersections of the lines drawn from each vertex to the midpoint of the opposite side. Its coordinates are simply the means of the coordinates of the vertices,

$$M_i = \tfrac{1}{3}(a_i + b_i + c_i) \qquad (i = 0, 1) \tag{21.3.9}$$

The centroid is also the point **M** where the areas $\mathcal{A}(\mathbf{abM})$, $\mathcal{A}(\mathbf{bcM})$, and $\mathcal{A}(\mathbf{caM})$ are all equal. In §21.7 we will be using a triangular mesh to interpolate a function. The significance of the centroid is that it is the point where a linearly interpolated function takes on the value that is the mean of the function values at the three vertices.

In fact, generalizing the idea of the centroid, any point **q** in the plane can be written as a linear combination of the three vertices **a**, **b**, **c**, with coefficients that sum to unity. These coefficients are called **q**'s *barycentric coordinates* and can be intuitively expressed in terms of the area formulas for triangles (see Figure 21.3.2). The equations are

$$\begin{aligned}\mathbf{q} &= \alpha\mathbf{a} + \beta\mathbf{b} + \gamma\mathbf{c} \\ \alpha &= \mathcal{A}(\mathbf{bcq})/\mathcal{A}(\mathbf{abc}) \\ \beta &= \mathcal{A}(\mathbf{caq})/\mathcal{A}(\mathbf{abc}) \\ \gamma &= \mathcal{A}(\mathbf{abq})/\mathcal{A}(\mathbf{abc})\end{aligned} \tag{21.3.10}$$

with, by construction,

$$\alpha + \beta + \gamma = 1 \tag{21.3.11}$$

The first line in equation (21.3.10) is thus equivalent to

$$\mathbf{q} = \mathbf{c} + \alpha(\mathbf{a} - \mathbf{c}) + \beta(\mathbf{b} - \mathbf{c}) \tag{21.3.12}$$

This can be viewed as the equation for a coordinate transformation, one that transforms from (α, β) coordinates to (q_0, q_1) coordinates. Evidently, since it is linear, its inverse — the formulas for α and β in equation (21.3.10) — must also be linear. But we knew this already, having remarked on the fact that the area formulas (21.3.1) are linear in all their coordinates, so linear in q_0 and q_1 in particular. Barycentric coordinates generalize to triangles in three or more dimensions in a useful way, as we will see below.

Note that α, β, or γ go to 1 as the point $\mathbf{q}$ approaches $\mathbf{a}$, $\mathbf{b}$, or $\mathbf{c}$, respectively; and that along any edge of the triangle (say $\overline{ab}$) the coefficient of the opposite vertex (here, γ) vanishes. The point $\mathbf{q}$ is inside the triangle $\triangle\mathbf{abc}$ if and only if α, β, and γ are all positive. In fact, this is a good way to test for a point's "insideness" in a triangle. (You can of course omit calculating the denominator area in this application.)

Barycentric coordinates are also useful when you want to pick a uniformly random point $\mathbf{q}$ inside $\triangle\mathbf{abc}$: First pick α and β as each uniformly random in $(0, 1)$. Next, if $\alpha + \beta > 1$, modify them both by $\alpha \leftarrow 1 - \alpha$ and $\beta \leftarrow 1 - \beta$. Finally, apply equation (21.3.12). The idea is that the first choice of α and β is random in the parallelogram spanned by two sides of the triangle; then, if it is on the wrong side of the diagonal, we move it to the correct side by a reflection.

21.3.1 Triangles in Three Dimensions

Our favorite triangle is still defined by the three points $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$, but these are now points in three dimensions, with coordinates (e.g., for $\mathbf{a}$) (a_0, a_1, a_2). The generalization of the signed area $\mathcal{A}$ (equation 21.3.1) is now a vector area $\vec{\mathcal{A}}$ whose direction is normal to the plane of the triangle and whose length is the area of the triangle. It is most easily written using a vector cross product, defined in three dimensions by

$$\begin{aligned}\mathbf{A} \times \mathbf{B} &= \begin{vmatrix} \hat{\mathbf{e}}_0 & \hat{\mathbf{e}}_1 & \hat{\mathbf{e}}_2 \\ A_0 & A_1 & A_2 \\ B_0 & B_1 & B_2 \end{vmatrix} \\ &= (A_1B_2 - A_2B_1)\hat{\mathbf{e}}_0 + (A_2B_0 - A_0B_2)\hat{\mathbf{e}}_1 + (A_0B_1 - A_1B_0)\hat{\mathbf{e}}_2\end{aligned} \tag{21.3.13}$$

where $\hat{\mathbf{e}}_0$, $\hat{\mathbf{e}}_1$, and $\hat{\mathbf{e}}_2$ are respectively the unit vectors $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$. Then we have (cf. equation 21.3.1)

$$\begin{aligned} 2\vec{\mathcal{A}}(\mathbf{abc}) &= (\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a}) \\ &= (\mathbf{c}-\mathbf{b})\times(\mathbf{a}-\mathbf{b}) \\ &= (\mathbf{a}-\mathbf{c})\times(\mathbf{b}-\mathbf{c}) \end{aligned} \tag{21.3.14}$$

You calculate the positive scalar area $\mathcal{A} \equiv |\vec{\mathcal{A}}|$ by the usual square-root sum of the squares of $\vec{\mathcal{A}}$'s three components; or you can instead use equation (21.3.3), with $d_{ab} = |\mathbf{a}-\mathbf{b}|$, etc.

Plane Defined by Triangle. A point $\mathbf{q}$ lies in the plane defined by $\triangle\mathbf{abc}$ if and only if the volume of the tetrahedron $\mathbf{abcq}$ is zero. The tetrahedral volume, in general, is given by

$$\begin{aligned} 6\mathcal{V} &= \begin{vmatrix} a_0 & a_1 & a_2 & 1 \\ b_0 & b_1 & b_2 & 1 \\ c_0 & c_1 & c_2 & 1 \\ q_0 & q_1 & q_2 & 1 \end{vmatrix} \\ &= (\mathbf{b}-\mathbf{a})\cdot[(\mathbf{c}-\mathbf{a})\times(\mathbf{q}-\mathbf{a})] \\ &= (\mathbf{c}-\mathbf{a})\cdot[(\mathbf{q}-\mathbf{a})\times(\mathbf{b}-\mathbf{a})] \\ &= (\mathbf{q}-\mathbf{a})\cdot[(\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a})] \end{aligned} \tag{21.3.15}$$

where "$\cdot$" signifies vector dot product. You can also cyclically permute $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ in the above equation, for a seemingly infinite number of variations of the same formula!

The volume $\mathcal{V}$ is signed and is positive if $\triangle\mathbf{abc}$ is counterclockwise when viewed from outside (side away from $\mathbf{q}$), that is, the right-hand rule gives an outward-pointing normal.

The last form in equation (21.3.15) is particularly nice, because setting it to zero gives the equation satisfied by any point $\mathbf{q}$ in the plane defined by $\triangle\mathbf{abc}$:

$$\mathbf{q}\cdot\mathbf{N} = D \tag{21.3.16}$$

with

$$\begin{aligned} &\mathbf{N} = (\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a}) \quad \text{(or cyclic permutation of } \mathbf{a}, \mathbf{b}, \mathbf{c}) \\ &D = \mathbf{a}\cdot\mathbf{N} \quad \text{(or, for that matter)} \quad = \mathbf{b}\cdot\mathbf{N} = \mathbf{c}\cdot\mathbf{N} \end{aligned} \tag{21.3.17}$$

We could also divide equation (21.3.16) by $|\mathbf{N}|$, in which case the vector on the left will be $\hat{\mathbf{N}} = \mathbf{N}/|\mathbf{N}|$, the unit vector normal to the plane, and $\hat{D} = D/|\mathbf{N}|$ will be the plane's distance from the origin.

With the same machinery, we can readily project any point $\mathbf{p}$ into a new point $\mathbf{b}'$ that lies in the plane of $\triangle\mathbf{abc}$:

$$\mathbf{p} \longrightarrow \mathbf{p}' = \mathbf{p} + \frac{[(\mathbf{a}-\mathbf{p})\cdot\mathbf{N}]\mathbf{N}}{|\mathbf{N}|^2} \tag{21.3.18}$$

where $\mathbf{N}$ is as above. For $\mathbf{a}$ in this formula, you can substitute $\mathbf{b}$, $\mathbf{c}$, or any other point in the plane.

We can project one triangle into the plane defined by another triangle by projecting its three points in turn. (This is a very common operation in rendering a triangulated three-dimensional model in the two-dimensional "camera plane" of your computer's screen.)

Barycentric Coordinates. Barycentric coordinates are valid in three dimensions for points $\mathbf{q}$ in the triangle's plane, and equation (21.3.10), in particular, still holds. To compute (α, β), one can in principle calculate the various $\mathcal{A}$'s from (21.3.14), but an easier equivalent calculation is

$$\begin{aligned} \alpha &= \frac{\mathbf{b}'^2(\mathbf{a}' \cdot \mathbf{q}') - (\mathbf{a}' \cdot \mathbf{b}')\,(\mathbf{b}' \cdot \mathbf{q}')}{\mathbf{a}'^2\mathbf{b}'^2 - (\mathbf{a}' \cdot \mathbf{b}')^2} \\ \beta &= \frac{\mathbf{a}'^2(\mathbf{b}' \cdot \mathbf{q}') - (\mathbf{a}' \cdot \mathbf{b}')\,(\mathbf{a}' \cdot \mathbf{q}')}{\mathbf{a}'^2\mathbf{b}'^2 - (\mathbf{a}' \cdot \mathbf{b}')^2} \end{aligned} \tag{21.3.19}$$

(compute identical denominators only once) where

$$\mathbf{a}' \equiv \mathbf{a} - \mathbf{c}, \qquad \mathbf{b}' \equiv \mathbf{b} - \mathbf{c}, \qquad \mathbf{q}' \equiv \mathbf{q} - \mathbf{c} \tag{21.3.20}$$

By the way, if $\mathbf{q}$ is not in the plane of $\triangle\mathbf{abc}$, you can still use equation (21.3.19). In that case, you get the (α, β) coordinates of the projected point in the plane. Also, notice what happens in the special case that $\triangle\mathbf{abc}$ is a right triangle, with right vertex $\mathbf{c}$, and with sides $\overline{ac}$ and $\overline{bc}$ of unit length, i.e., $d_{ac} = d_{bc} = 1$. Then the coordinate transformations, in both directions, are simply

$$\begin{aligned} \mathbf{q} &= \mathbf{c} + \alpha(\mathbf{a} - \mathbf{c}) + \beta(\mathbf{b} - \mathbf{c}) \\ [\alpha, \beta] &= [(\mathbf{a} - \mathbf{c}) \cdot (\mathbf{q} - \mathbf{c}), (\mathbf{b} - \mathbf{c}) \cdot (\mathbf{q} - \mathbf{c})] \end{aligned} \tag{21.3.21}$$

In other words, we project into an orthonormal coordinate system in the plane by a simple change of origin (to $\mathbf{c}$) and dot products with the two "axes" $\mathbf{a} - \mathbf{c}$ and $\mathbf{b} - \mathbf{c}$.

Frequently, barycentric coordinates are the coordinates of choice for operations in a plane in three dimensions that is (or can be) specified by a triangle. A trivial example is that we can test whether a projected point $\mathbf{p}'$ is inside or outside of $\triangle\mathbf{abc}$ by using equation (21.3.19) (or, if applicable, 21.3.21) to get α and β, and then checking that α, β, and $\gamma = 1 - \alpha - \beta$ are all positive.

Angle Between Two Triangles. The dihedral angle between two triangles (with a common edge, say) is the same as the angle between the normal vectors of the two triangles. The normal vectors are given by the vector area formula (21.3.14). The angle is best computed using equation (21.4.13), in the next section.

CITED REFERENCES AND FURTHER READING:

Bowyer, A. and Woodwark, J. 1983, *A Programmer's Geometry* (London: Butterworths), Chapter 4.

Schneider, P.J. and Eberly, D.H. 2003, *Geometric Tools for Computer Graphics* (San Francisco: Morgan Kaufmann), §3.5 and Appendix C.

López-López, F.J. 1992, "Triangles Revisited," in *Graphics Gems III*, Kirk, D., ed. (Cambridge, MA: Academic Press).

Glassner, A.S. 1990, "Useful 3D Geometry," in *Graphics Gems*, Glassner, A.S., ed. (San Diego: Academic Press).

21.4 Lines, Line Segments, and Polygons

A *line* is defined by any two points through which it passes. Call them **a** and **b**. As in §21.1, the points can be two-dimensional, if the domain of interest is a plane, or three-dimensional (or higher), if the line is embedded in a higher-dimensional space. For now, consider only the two-dimensional case.

Parametrically, any point **c** that lies on the line defined by **a** and **b** must be a linear combination of those two points. One way to write this is

$$\mathbf{c} = \mathbf{a} + s(\mathbf{b} - \mathbf{a}) \qquad (-\infty < s < \infty) \tag{21.4.1}$$

where s is a parameter along the line. The chosen normalization is to make $s = 0$ at **a** and $s = 1$ at **b**. The part of the line between **a** and **b** has $0 \le s \le 1$ and is a *line segment*, denoted $\overline{\mathbf{ab}}$. The whole line is denoted $\overleftrightarrow{\mathbf{ab}}$.

The easiest way to get the equation satisfied by all points **c** on the line $\overleftrightarrow{\mathbf{ab}}$ is to take the vector cross product of equation (21.4.1) with $(\mathbf{b} - \mathbf{a})$ on the right. Using the fact that the cross product of any vector with itself is zero, we get

$$\mathbf{c} \times (\mathbf{b} - \mathbf{a}) = \mathbf{a} \times \mathbf{b} \tag{21.4.2}$$

or writing out the components,

$$c_0(b_1 - a_1) - c_1(b_0 - a_0) = a_0 b_1 - a_1 b_0 \tag{21.4.3}$$

which is indeed a linear relation between the coordinates c_0 and c_1. While it is tempting to divide this equation by $b_0 - a_0$ to get an equation in that old familiar high school form "$y = mx + b$," one should often resist that temptation since, as written, equation (21.4.3) remains valid for the case of a vertical line, when $b_0 - a_0 = 0$.

Intersection of Two Lines. In the plane, two lines $\overleftrightarrow{\mathbf{ab}}$ and $\overleftrightarrow{\mathbf{xy}}$ most always intersect. We can solve for the point of intersection by equating the two lines' parametric forms,

$$\mathbf{a} + s(\mathbf{b} - \mathbf{a}) = \mathbf{x} - t(\mathbf{y} - \mathbf{x}) \tag{21.4.4}$$

and then solving the two equations (components 0 and 1) for the two unknowns s and t. The result is

$$\begin{aligned} s &= \frac{(\mathbf{x} - \mathbf{y}) \times (\mathbf{a} - \mathbf{x})}{(\mathbf{b} - \mathbf{a}) \times (\mathbf{x} - \mathbf{y})} = \frac{(x_0 - y_0)(a_1 - x_1) - (x_1 - y_1)(a_0 - x_0)}{(b_0 - a_0)(x_1 - y_1) - (b_1 - a_1)(x_0 - y_0)} \\ t &= \frac{(\mathbf{a} - \mathbf{x}) \times (\mathbf{b} - \mathbf{a})}{(\mathbf{b} - \mathbf{a}) \times (\mathbf{x} - \mathbf{y})} = \frac{(a_0 - x_0)(b_1 - a_1) - (a_1 - x_1)(b_0 - a_0)}{(b_0 - a_0)(x_1 - y_1) - (b_1 - a_1)(x_0 - y_0)} \end{aligned} \tag{21.4.5}$$

Of course, the special case of parallel lines with no intersection is indicated by the vanishing of the denominators.

All those cross products might make you think that equation (21.4.5) has a geometrical interpretation. Indeed so. In Figure 21.4.1, the lines intersect at **o**. Segment $\overline{\mathbf{xo}}$ is therefore just $\overline{\mathbf{xy}}$ scaled by t, while $\overline{\mathbf{ao}}$ is similarly $\overline{\mathbf{ab}}$ scaled by s. The area of $\triangle\mathbf{oxa}$ is therefore given (cf. equation 21.3.1) by

$$2\mathcal{A}(\mathbf{oxa}) = st\ (\mathbf{x} - \mathbf{y}) \times (\mathbf{a} - \mathbf{b}) \tag{21.4.6}$$

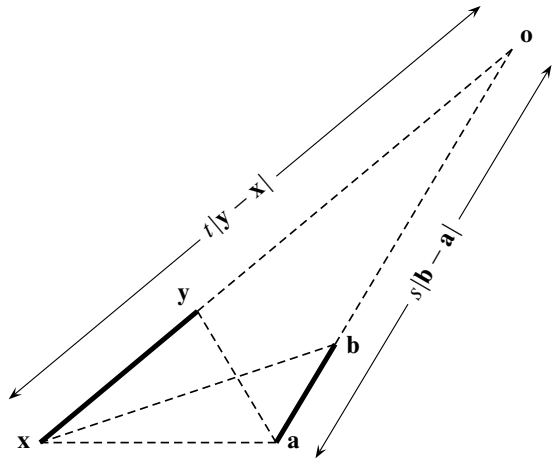


Figure 21.4.1. Geometrical construction that yields the intersection point of two lines in terms of ratios of triangle areas. See text for details.

By linearity of a triangle's area with its height (holding the base fixed), we also have

$$\mathcal{A}(\mathbf{oxa})/\mathcal{A}(\mathbf{yxa}) = t \qquad \mathcal{A}(\mathbf{oxa})/\mathcal{A}(\mathbf{bxa}) = s \tag{21.4.7}$$

Equation (21.4.5) follows immediately from these relations and equation (21.3.1).

Point-to-Line Distance. What is the perpendicular distance d from an arbitrary point $\mathbf{q}$ to the line $\overleftrightarrow{\mathbf{ab}}$ that passes through points $\mathbf{a}$ and $\mathbf{b}$? Evidently d is the height of $\triangle\mathbf{abq}$ when its base is the segment $\overline{\mathbf{ab}}$. Therefore, from the schoolbook "one-half base times height" formula,

$$d = \frac{2\mathcal{A}(\mathbf{abq})}{|\mathbf{a}-\mathbf{b}|} = \frac{(q_0-b_0)(a_1-b_1)-(q_1-b_1)(a_0-b_0)}{\sqrt{(a_0-b_0)^2+(a_1-b_1)^2}} \tag{21.4.8}$$

Note that d is signed, positive if it is to the left of the directed line from $\mathbf{a}$ to $\mathbf{b}$, negative if it is to the right, and a good segue to our next topic.

21.4.1 Line Segment Intersections and "Left-Of" Relations

You can use equation (21.4.5) to test whether two line segments, $\overline{\mathbf{ab}}$ and $\overline{\mathbf{xy}}$, intersect: Calculate s and t and then check if they are both in the range $(0, 1)$. (To keep our discussion brief, we won't say much here, or in what follows, about the various degenerate cases where s or t or both are exactly 0 or 1. These are straightforward to figure out if your application so demands.)

A related, but slightly different, approach is to use the fact that the formulas for triangle areas given in equation (21.3.1) are signed. This means that, as equivalent statements, we have

$$\mathcal{A}(\mathbf{abc}) > 0 \iff \left[\begin{array}{l}\mathbf{c} \text{ is to the } \textit{left} \text{ of line } \overleftrightarrow{\mathbf{ab}} \\ \text{when it is traversed in} \\ \text{the direction from } \mathbf{a} \text{ to } \mathbf{b}\end{array}\right] \tag{21.4.9}$$

while $\mathcal{A}(\mathbf{abc}) < 0$ implies that $\mathbf{c}$ is to the right of the same line. We refer to either statement in (21.4.9) as a *left-of relation*.

A necessary and sufficient condition for two segments $\overline{\mathbf{ab}}$ and $\overline{\mathbf{xy}}$ to intersect is that **x** and **y** be on opposite sides of $\overleftrightarrow{\mathbf{ab}}$, *and* **a** and **b** be on opposite sides of $\overleftrightarrow{\mathbf{xy}}$. (We again omit discussion of the various special cases of collinearity.) This test, using the triangle area formulas in equation (21.3.1), involves evaluating four left-of relations, each computationally a cross product, which is just slightly more work than computing s and t (which share a denominator). However, you can sometimes use the same cross products, once computed, in other parts of your calculation. So, it is often a toss-up whether to use the "s, t" method or the "left-of" method — you should consider both.

Fig.	△abx	△aby	△xya	△xyb	Intersection	Hull
1	−	+	+	−	$\overline{\text{ab}} \times \overline{\text{xy}}$	□axby
2	+	−	−	+	$\overline{\text{ab}} \times \overline{\text{xy}}$	□aybx
3	+	−	−	−	$\overrightarrow{\text{ab}} \times \overline{\text{xy}}$	△ayx
4	−	+	−	−	$\overrightarrow{\text{ba}} \times \overline{\text{xy}}$	△byx
5	+	−	+	+	$\overrightarrow{\text{ba}} \times \overline{\text{xy}}$	△bxy
6	−	+	+	+	$\overrightarrow{\text{ab}} \times \overline{\text{xy}}$	△axy
7	−	−	−	+	$\overrightarrow{\text{yx}} \times \overline{\text{ab}}$	△yba
8	−	−	+	−	$\overrightarrow{\text{xy}} \times \overline{\text{ab}}$	△xba
9	+	+	−	+	$\overrightarrow{\text{xy}} \times \overline{\text{ab}}$	△xab
10	+	+	+	−	$\overrightarrow{\text{yx}} \times \overline{\text{ab}}$	△yab
11	−	−	−	−	external	□ayxb
12	+	+	−	−	external	□abxy
13	+	+	+	+	external	□abxy
14	−	−	+	+	external	□axyb
	−	+	−	+	Not possible!	
	+	−	+	−	Not possible!	

Table 21.4.1. Relationship between two line segments classified by the signs of the areas of various triangles. Refer to Figure 21.4.2 for an illustration of each case.

Table 21.4.1 enumerates the 16 cases that you get if you compute all four possible "left-of" tests for two line segments. (Actually, only 14 are geometrically possible!) The table classifies each possibility as to whether the segments intersect (intersection denoted by ×, not to be confused with vector cross product!), whether the unidirectional extension of one segment (a *ray*) intersects the other segment, or (lastly) whether an ordinary line intersection occurs external to both segments. Also shown for each case is the outer *hull* of the two segments (the smallest triangle or quadrilateral enclosing both segments) and how it is traversed in clockwise order. Figure 21.4.2 shows an example of each possibility.

You can use the table to find combinations that test for specific circumstances.

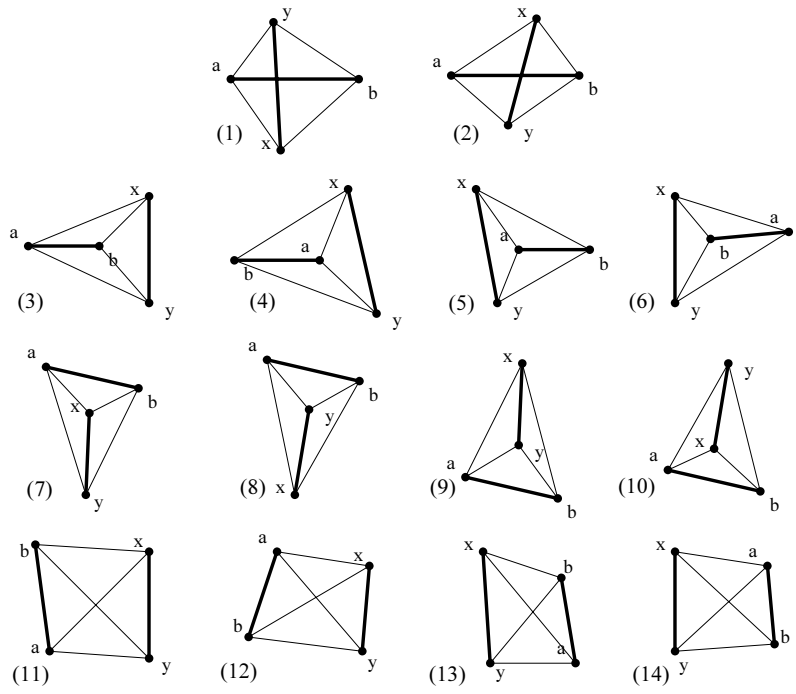


Figure 21.4.2. Two line segments, ab and xy, define four triangles ($\triangle abx$, $\triangle aby$, $\triangle xya$, and $\triangle xyb$), each of which can have positive or negative area. Of the 16 combinations of signs, 14 (shown here) correspond to to possible intersection relationships between the line segments or their extensions as rays.

For example, if you need a test for whether the ray $\overrightarrow{\mathbf{ab}}$ intersects the segment $\overline{\mathbf{xy}}$ (including the possibility of the segments intersecting), you examine rows 1, 2, 3, and 6 in the table and read off a test that involves just three left-of relations:

$$\mathcal{A}(\mathbf{abx})\mathcal{A}(\mathbf{aby}) < 0 \qquad \text{and} \qquad \mathcal{A}(\mathbf{aby})\mathcal{A}(\mathbf{xya}) > 0 \tag{21.4.10}$$

Of course there are exactly equivalent tests using s and t, for this example (with s and t as in equation 21.4.4) $s > 0$ and $0 < t < 1$.

Angle Between Two Vectors. Suppose that $\mathbf{U}$ and $\mathbf{V}$ are difference vectors along each of two lines, and that θ is the angle between the lines (measured from $\mathbf{U}$ to $\mathbf{V}$). In the previous notation, $\mathbf{U} = \mathbf{y} - \mathbf{x}$ and $\mathbf{V} = \mathbf{b} - \mathbf{a}$. Elementary vector analysis tells us that

$$\begin{aligned} \mathbf{U} \cdot \mathbf{V} &= U_0 V_0 + U_1 V_1 = |\mathbf{U}||\mathbf{V}| \cos(\theta) \\ \mathbf{U} \times \mathbf{V} &= U_0 V_1 - U_1 V_0 = |\mathbf{U}||\mathbf{V}| \sin(\theta) \end{aligned} \tag{21.4.11}$$

Many people try to get θ by using one or the other of the above relations, computing the vector norms and taking an inverse cosine or inverse sine. Big mistake! Not only are there quadrant ambiguities in the inverse trig functions, but there are also angles near the flat extrema of the sine and cosine functions where you can lose up to half of your significant figures in the answer. Not to mention the need to calculate square roots for the norms! The better approach is

$$\theta = \texttt{atan2}(\mathbf{U} \times \mathbf{V}, \mathbf{U} \cdot \mathbf{V}) = \text{atan2}(U_0 V_1 - U_1 V_0, U_0 V_0 + U_1 V_1) \tag{21.4.12}$$

where `atan2()` is C's or C++'s quadrant-sensitive arctangent function. That function allows either of its arguments to be zero and returns a value in the range $-\pi/2$ to $\pi/2$. (An identical function exists in Fortran and most other languages.)

21.4.2 Lines in Three Dimensions

The immediate generalization of equation (21.4.12) to three-dimensional space gives the angle between two 3-vectors,

$$\theta = \texttt{atan2}\,(|\mathbf{U} \times \mathbf{V}|, \mathbf{U} \cdot \mathbf{V}) \tag{21.4.13}$$

Note the occurrence of the modulus of the vector cross product, which requires taking a square root.

Brevity constraints allow us to say only a little more about lines in three-dimensional space. The parameterization

$$\mathbf{c} = \mathbf{a} + s\,\mathbf{v} \qquad (-\infty < s < \infty) \tag{21.4.14}$$

(equation 21.4.1 with $\mathbf{v} \equiv \mathbf{b} - \mathbf{a}$) still works, with $\mathbf{a}$, $\mathbf{v}$, and $\mathbf{c}$ now points in 3-space. The parameter s at which a line intersects a plane specified by $\mathbf{N}$ and D (see equation 21.3.16) is given by

$$s = \frac{D - \mathbf{a} \cdot \mathbf{N}}{\mathbf{v} \cdot \mathbf{N}} \tag{21.4.15}$$

with the denominator vanishing if the line is parallel to the plane.

The closest approach of a line to a point $\mathbf{q}$ occurs when

$$s = \frac{(\mathbf{q} - \mathbf{a}) \cdot \mathbf{v}}{|\mathbf{v}|^2} \tag{21.4.16}$$

You can also use this to see whether a line intersects a sphere in 3-space: Calculate the closest point on the line to the sphere's center, and then check if the distance is less than the sphere's radius (or compare squares of distances to avoid the square root).

Two lines, call them $\mathbf{a} + s\,\mathbf{v}$ and $\mathbf{x} + t\,\mathbf{u}$, will not, in general, share a common point; rather, they will be *skew* to one another. However, their points of closest approach can be calculated as [2]

$$s = \frac{\det\{(\mathbf{a} - \mathbf{x}), \mathbf{u}, \mathbf{u} \times \mathbf{v})\}}{|\mathbf{u} \times \mathbf{v}|^2} \qquad t = \frac{\det\{(\mathbf{a} - \mathbf{x}), \mathbf{v}, \mathbf{u} \times \mathbf{v})\}}{|\mathbf{u} \times \mathbf{v}|^2} \tag{21.4.17}$$

where det is the 3×3 determinant whose columns are the indicated 3-vectors. The denominator vanishes if the lines are parallel. If you really must check for an actual intersection, plug these values for s and t into the parametric forms for each line, and check whether the distance between two points thus obtained is less than some roundoff tolerance.

A common operation in computer graphics is to test whether a line intersects a triangle in three dimensions. To do this with methods already discussed, use equation (21.3.17) to get $\mathbf{N}$ and D for the triangle's plane. Then use equations (21.4.14) and (21.4.15) to get the line's intersection with that plane. Finally, use equation (21.3.19) to get the barycentric coordinates α and β of the intersection. If α, β, and $\gamma \equiv 1 - \alpha - \beta$ are all positive, then the intersection is inside the triangle. See [4,1] for various ways to streamline this procedure.

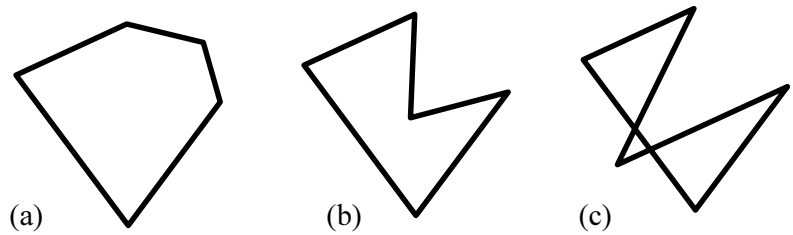


Figure 21.4.3. Polygons are classified as simple if they have no intersecting edge, as in (a) and (b), or complex (c) otherwise. Simple polygons are either convex (a) or concave (b).

21.4.3 Polygons

We define a *polygon* as a vector of N points (*vertices*), numbered from 0 to $N-1$, and the N directed line segments that connect them in cyclic order, that is 0 to 1, 1 to 2, ..., $N-2$ to $N-1$, and (importantly!) $N-1$ to 0. (In some formulas below we will use the convention that vertex N is to be taken as meaning vertex 0.)

We consider two polygons to be the same if they differ only by a cyclical renumbering of the points, so that all their line segments are the same. However, if we reverse the order of traversing the points, we consider the resulting polygon to be different. (For example, the sign of its area will change.) If the boundary of a region cannot be traversed by a single cyclical vector (e.g., the region between an outer square and an enclosed triangle), we don't call it a polygon; other conventions are of course also possible.

With the definition given, it is useful to classify a polygon as either *simple*, meaning that none of its N line segments intersect, or *complex* if there are one or more segment intersections. We classify simple polygons according to whether they are convex or concave. A convex polygon can be defined either by (i) all $N(N-1)/2$ line segments connecting two vertices lie in its interior (or on its boundary), or (ii) its exterior angles all have the same sign (zeros allowed). Whichever property is taken as the definition, the other becomes a theorem. Figure 21.4.3 shows examples of the three types.

For simple polygons, the sum of the exterior angles is always $\pm 2\pi$. That is, you turn through exactly one circle in driving around the polygon. If the polygon is concave, then the sign of the exterior angles must be taken into account when doing the sum. This is shown in Figure 21.4.4. The sign of the 2π is positive for a counterclockwise (CCW) traversal, negative for clockwise (CW).

Complex (that is, self-intersecting) polygons can also have exterior angles that sum to 2π, as the polygon in Figure 21.4.3(c), so the exterior angles do not provide, in general, any magical way of finding intersections. However, one small bit of magic does exist: If the exterior angles of a polygon are all of one sign (or zero), *and* if they sum to $\pm 2\pi$, then the polygon is both simple and convex. This provides a very rapid way to test for the simple-and-convex case, but it does not distinguish between simple-concave and complex polygons. Doing so requires a detailed check for intersecting edges (which we will implement in code below).

Winding Number. If you sit on a point **p** in the plane, and watch someone drive around a polygon, then they will drive around *you* some net integer number of times (with the usual sign convention, CCW being positive). This is the polygon's *winding number with respect to a point*. For simple polygons, the winding number is 1 for

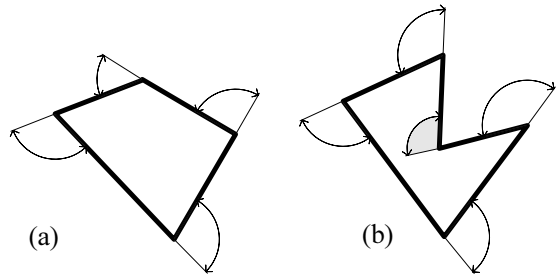


Figure 21.4.4. The exterior angles of simple polygons sum to one full circle. (a) If the polygon is convex, all the angles have the same sign. (b) If the polygon is concave, one or more angles (here, the shaded angle) has opposite sign.

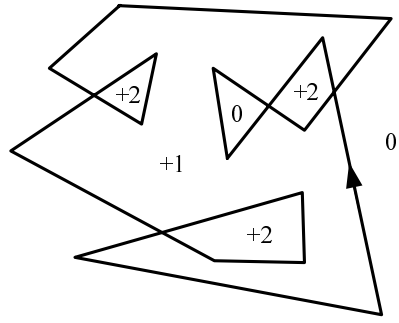


Figure 21.4.5. Complex polygon with different winding numbers (indicated by integers) around points in different regions. The total winding number of the polygon (sum of exterior angles divided by 2π) is 3, a value not realized in any single region. Note that interior regions can have winding number 0.

points inside a CCW polygon, -1 for points inside a CW polygon, and 0 for points outside. For complex polygons, however, there are no simple rules. Figure 21.4.5 shows a complicated case. Note that interior regions of a complex polygon can have winding number 0, so a point's winding number (alone) does not determine whether it is inside or outside a complex polygon. Note also that the sum of a polygon's exterior angles, divided by 2π, is not necessarily the winding number of that polygon with respect to any point in the plane.

Doubtless the *worst* way to compute a polygon's winding number with respect to a point $\mathbf{q}$ is to add up the N incremental angles between $\mathbf{q}$ and consecutive vertices $\mathbf{p}_i$ of the polygon, that is,

$$\text{W.N.}(\mathbf{q}) = \frac{1}{2\pi} \sum_{i=0}^{N-1} \angle(\mathbf{p}_{i+1}\mathbf{q}\mathbf{p}_i) \tag{21.4.18}$$

(with the usual convention $\mathbf{p}_N \equiv \mathbf{p}_0$). Even using the trick in equation (21.4.12) to get the angles, there is an enormous amount of unnecessary computation in this approach.

Instead, we can observe that if a polygon winds M times around $\mathbf{q}$, then its edges must cross any ray from $\mathbf{q}$ to infinity a net of exactly M times, where ray crossings in the CCW direction are counted as positive, CW as negative. In particular, if we take the ray to be the horizontal ray to the right of $\mathbf{q}$, we can immediately

reject edges that fail to cross the horizontal line that contains $\mathbf{q}$, and then check for the ray crossing (and its sign) with a single "left-of" test [5]. These ideas are embodied in the following routine.

polygon.h

```
Int polywind(const vector< Point<2> > &vt, const Point<2> &pt) {
```

Return the winding number of a polygon (specified by a vector of vertex points `vt`) around an arbitrary point `pt`.

```
    Int i,np, wind = 0;
    Doub d0,d1,p0,p1,pt0,pt1;
    np = vt.size();
    pt0 = pt.x[0];
    pt1 = pt.x[1];
    p0 = vt[np-1].x[0];                    Save last vertex as "previous" to first.
    p1 = vt[np-1].x[1];
    for (i=0; i<np; i++) {                 Loop over edges.
        d0 = vt[i].x[0];
        d1 = vt[i].x[1];
        if (p1 <= pt1) {
            if (d1 > pt1 &&                Upward-crossing edge. Is pt to its left?
                (p0-pt0)*(d1-pt1)-(p1-pt1)*(d0-pt0) > 0) wind++;
        }
        else {
            if (d1 <= pt1 &&               Downward-crossing edge. Is pt to its right?
                (p0-pt0)*(d1-pt1)-(p1-pt1)*(d0-pt0) < 0) wind--;
        }
        p0=d0;                             Current vertex becomes previous one.
        p1=d1;
    }
    return wind;
}
```

Is there a similarly efficient way to find the total winding number of a polygon $\mathbf{p}_i$ $(i = 0, \ldots, N-1)$, that is, the sum of its exterior angles divided by 2π? Yes. Consider the derived polygon whose vertex points $\mathbf{q}_i$ are given by the vector differences

$$\mathbf{q}_i = \mathbf{p}_{i+1} - \mathbf{p}_i \qquad (i = 0, \ldots, N-1) \tag{21.4.19}$$

Then the winding number of this derived polygon *around the origin* is just the total winding number of the original polygon. (Draw a picture if this isn't immediately obvious to you.) The routine `polywind()` can be used for the computation.

Point Inside Polygon. How can you tell whether an arbitrary point $\mathbf{q}$ is inside or outside a polygon [5]? Let us first assume that your polygon is known to be simple. For simple polygons, two commonly used approaches are the "winding number method" and the "Jordan curve theorem method." However, when these are each implemented efficiently, they become virtually identical!

The winding number method is simply to compute the winding number of the polygon around the point (e.g., using `polywind()`, above). If the answer is ± 1, then the point is inside the polygon. If it is zero, it is outside. Any other answer indicates that the polygon wasn't simple as assumed.

The Jordan curve theorem method observes that any ray from the point to infinity will cross the polygon an odd number of times if the point is inside, or an even number of times if it is outside [Figure 21.4.6(a)]. If we implemented this in code, it would be almost identical to the code in `polywind` except for one detail: Instead of incrementing or decrementing a counter at each ray crossing (according to the direction of the crossing), we would always increment it. Then, at the end, we would

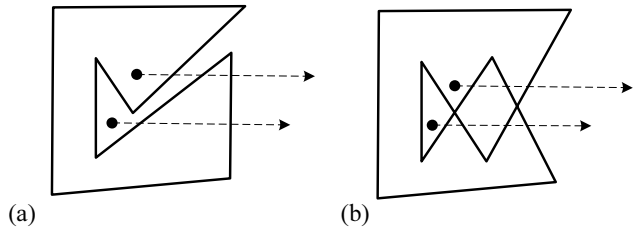


Figure 21.4.6. Is a point inside a polygon? (a) For a simple polygon, either the winding number, or the Jordan curve theorem (even or odd number of crossings of a ray) can be used. (b) For complex polygons, there is no simple test.

check whether the counter is even or odd. But if `polywind` as written returns 0, it *must* have encountered the same number of increments as decrements, hence an even number of crossings. And if it returns ± 1 (the only other possible value for a simple polygon), it must similarly have encountered an odd number. So the two methods are really the same.

What if your polygon is not simple? As Figure 21.4.6(b) illustrates, you are in deep waters. Both the winding number method and the Jordan curve theorem method will say that the upper point in the figure is inside the complex polygon shown, and this seems intuitively correct. However, both methods will say that the lower point is *outside* the polygon. Indeed, there are some self-consistent ways of defining "insideness" for complex polygons that make this the case. The result is so counterintuitive, however, as to be useless in most practical applications. It is generally better just to avoid using the idea of "insideness" with complex polygons.

Classification of Polygons. We are now in a position to combine several of the ideas already introduced into a function that classifies any polynomial as either simple or complex, and (if it is simple) whether it is convex or concave, and whether it is CCW (total winding number 1) or CW (total winding number -1).

polygon.h

```
Int ispolysimple(const vector< Point<2> > &vt) {
```
Classifies a polygon specified by a vector of vertex points `vt`. Returns 0 if the polygon is complex (has intersecting edges). Returns ± 1 if it is simple and convex. Returns ± 2 if it is simple and concave. The sign of the returned value indicates whether the polygon is CCW (+) or CW (−).

```
    Int i,ii,j,jj,np,schg=0,wind=0;          Initialize sign change and winding number.
    Doub p0,p1,d0,d1,pp0,pp1,dd0,dd1,t,tp,t1,t2,crs,crsp=0.0;
    np = vt.size();
    p0 = vt[0].x[0]-vt[np-1].x[0];
    p1 = vt[0].x[1]-vt[np-1].x[1];
    for (i=0,ii=1; i<np; i++,ii++) {         Loop over edges.
        if (ii == np) ii = 0;
        d0 = vt[ii].x[0]-vt[i].x[0];
        d1 = vt[ii].x[1]-vt[i].x[1];
        crs = p0*d1-p1*d0;                   Cross product at this vertex.
        if (crs*crsp < 0) schg = 1;          Sign change (i.e., concavity) found.
        if (p1 <= 0.0) {                     Winding number logic as in polywind.
            if (d1 > 0.0 && crs > 0.0) wind++;
        } else {
            if (d1 <= 0.0 && crs < 0.0) wind--;
        }
        p0=d0;
        p1=d1;
        if (crs != 0.0) crsp = crs;          Save previous cross product only if it has a
    }                                            sign!
```

```
    if (abs(wind) != 1) return 0;          Can already conclude polygon is complex.
    if (schg == 0) return (wind>0? 1 : -1);    Polygon is simple and convex.
    Drat, we've exhausted all the quick tricks and now have to check all pairs of edges for
    intersections:
    for (i=0,ii=1; i<np; i++,ii++) {
        if (ii == np) ii=0;
        d0 = vt[ii].x[0];
        d1 = vt[ii].x[1];
        p0 = vt[i].x[0];
        p1 = vt[i].x[1];
        tp = 0.0;
        for (j=i+1,jj=i+2; j<np; j++,jj++) {
            if (jj == np) {if (i==0) break; jj=0;}
            dd0 = vt[jj].x[0];
            dd1 = vt[jj].x[1];
            t = (dd0-d0)*(p1-d1) - (dd1-d1)*(p0-d0);
            if (t*tp <= 0.0 && j>i+1) {        First loop is only to compute starting tp,
                pp0 = vt[j].x[0];                  hence test on j.
                pp1 = vt[j].x[1];
                t1 = (p0-dd0)*(pp1-dd1) - (p1-dd1)*(pp0-dd0);
                t2 = (d0-dd0)*(pp1-dd1) - (d1-dd1)*(pp0-dd0);
                if (t1*t2 <= 0.0) return 0;    Found an intersection, so done.
            }
            tp = t;
        }
    }
    return (wind>0? 2 : -2);              No intersections found, so simple concave.
}
```

When `ispolysimple` finds that the quick indicators are not enough, and that it needs to check all pairs of edges for intersections, it does so by the obvious $O(N^2)$ method of two nested loops. For small N, say less than 10, this is likely as fast as any other strategy. If you are dealing with large numbers of large-N polygons, however, you will want to substitute a method with better scaling in N. One way, using the code from §21.8, would be to define a class for segments with a `collides()` method, then store the segments into a QO tree one at a time, looking for collisions at each step. (Don't forget that adjacent edges of a simple polynomial are allowed to "collide" at their shared vertex.)

Area of Polygons. As a next topic, let us turn to the *area* of a polygon. The (signed) area of a polygon is the sum of the areas of each of its regions weighted by that region's winding number. For simple polygons the area is thus what you would expect geometrically, except that its sign will be negative for a polygon traversed CW rather than CCW. (We previously saw this in the special case of triangles.) For a complex polygon like that shown in Figure 21.4.5, the answer is less intuitive (and generally less useful) since some regions, such as the interior region with winding number 0, are not counted at all, while others are counted (in this case) twice.

The great advantage of this definition of area, however, is that it results in a simple expression for the area that applies to both simple and complex polygons. Let x_i and y_i be, respectively, the 0 and 1 coordinates of the polygon's vertex $\mathbf{p}_i$, and let $\mathcal{A}$ be the polygon's area. Then, in three equivalent forms,

$$2\,\mathcal{A} = \sum_{i=0}^{N-1} x_i\, y_{i+1} - x_{i+1}\, y_i$$

$$= \sum_{i=0}^{N-1} (x_{i+1} + x_i)(y_{i+1} - y_i)$$
$$= \sum_{i=0}^{N-1} x_i (y_{i+1} - y_{i-1}) \tag{21.4.20}$$

Evaluation of any of these forms takes just one loop over the polygon's vertices. (These formulas go back as far as Meister in 1769 and Gauss in 1795.)

Although we won't derive equation (21.4.20) in detail, the middle form does have an intuitive interpretation. It sums the areas of trapezoids each with two points on the y-axis ($x = 0$) at y_i and y_{i+1}, and with the other two points the points on the polygon at these y's. In going around the polygon, negative-area trapezoids subtract from positive-area ones so as to leave just the area inside.

Interestingly, there are very similar formulas for the x and y coordinates of the centroid or center-of-mass of an arbitrary polygon [3],

$$\overline{x} = \frac{1}{6} \sum_{i=0}^{N-1} (x_{i+1} + x_i)(x_i y_{i+1} - x_{i+1} y_i)$$
$$\overline{y} = \frac{1}{6} \sum_{i=0}^{N-1} (y_{i+1} + y_i)(x_i y_{i+1} - x_{i+1} y_i) \tag{21.4.21}$$

Note the common subexpressions with equation (21.4.20), so that it is efficient to calculate the area and centroid position together.

Finally, a couple of polygon tidbits for your edification or amusement:

- If two simple polygons have the same area, then the first can be cut into a finite number of polygonal pieces that can be reassembled into the second. This is known as the Bolyai-Gerwien theorem. (The corresponding statement about polyhedra in three dimensions, "Hilbert's Third Problem," was proved false by Dehn in 1900.)
- The regular polygon with N sides is constructible with a compass and straightedge if the factorization of N contains only the factors 2, 3, 5, 17, 257, 257, and 65537 (whose odd members are the *Fermat primes*), with each odd factor occurring at most once. It is not known whether any other N-gons are also constructible; but, if so, then their N must contain a factor at least as large as $2^{2^{33}} + 1$. The product of the known Fermat primes, which is perforce the largest known constructible polygon with an odd number of sides, is $2^{32} - 1 = 4294967295$, a number well known to computer trolls as the largest positive 32-bit integer. Go figure.

CITED REFERENCES AND FURTHER READING:

Bowyer, A. and Woodwark, J. 1983, *A Programmer's Geometry* (London: Butterworths).

Schneider, P.J. and Eberly, D.H. 2003, *Geometric Tools for Computer Graphics* (San Francisco: Morgan Kaufmann), §11.1.2[1]

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 2.

O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press), §7.4.

Goldman, R. 1990, "Intersection of Two Lines in Three-Space," in *Graphics Gems*, Glassner, A.S., ed. (San Diego: Academic Press).[2]

Bashein, G. and Detmer, P.R. 1994, "Centroid of a Polygon," in *Graphics Gems IV*, Heckbert, P.S., ed. (Cambridge, MA: Academic Press).[3]

Sunday, D. 2007+, at `http://softsurfer.com/algorithm_archive.htm`.[4]

Haines, E. 1994, "Point in Polygon Strategies," in *Graphics Gems IV*, Heckbert, P.S., ed. (Cambridge, MA: Academic Press).[5]

Wikipedia 2007+, "Polygon," at `http://en.wikipedia.org`.

21.5 Spheres and Rotations

The surface of the Earth is called a 2-sphere by topologists, but a 3-sphere by geometers; so the term n-sphere is somewhat unclear. We'll say "sphere in n dimensions" to avoid any ambiguity. (For Earth, $n = 3$.) *Sphere* refers to the surface, *ball* to the interior volume.

A sphere of radius r in n dimensions, centered on the origin, is the locus of points for which

$$x_0^2 + \cdots + x_{n-1}^2 = r^2 \tag{21.5.1}$$

Points on the sphere in n dimensions can be specified by $n-1$ angular coordinates, roughly the analogs of latitude and longitude on the sphere in three dimensions,

$$\begin{aligned} x_0 &= r\cos\psi_0 \\ x_1 &= r\sin\psi_0\cos\psi_1 \\ &\cdots \\ x_{n-2} &= r\sin\psi_0\sin\psi_1\cdots\cos\psi_{n-2} \\ x_{n-1} &= r\sin\psi_0\sin\psi_1\cdots\sin\psi_{n-2} \end{aligned} \tag{21.5.2}$$

All the angles except the last have the range

$$0 \le \psi_i \le \pi, \qquad i = 0, \ldots, n-3 \tag{21.5.3}$$

i.e., are "latitude-like." The last angle is "longitude-like,"

$$0 \le \psi_{n-2} \le 2\pi \tag{21.5.4}$$

The surface area S_n of the sphere in n dimensions has a simple recurrence,

$$\begin{aligned} S_1 &= 2 && \text{(two points)} \\ S_2 &= 2\pi r && \text{(circumference of circle)} \\ S_n &= \frac{2\pi r^2}{n-2} S_{n-2}, && n > 2 \end{aligned} \tag{21.5.5}$$

The volume V_n of the n-dimensional ball is equal to r/n times the area of the enclosing sphere in n dimensions, and also has a simple recurrence,

$$\begin{aligned} V_1 &= 2r && \text{(length of line)} \\ V_2 &= \pi r^2 && \text{(area of circle)} \\ &\cdots \\ V_n &= \frac{r}{n} S_n = \frac{2\pi r^2}{n} V_{n-2} \end{aligned} \tag{21.5.6}$$

Closed-form formulas require a gamma function,

$$\begin{aligned} S_n &= \frac{2\pi^{n/2}}{\Gamma(\frac{1}{2}n)} r^{n-1} \\ V_n &= \frac{2\pi^{n/2}}{n\Gamma(\frac{1}{2}n)} r^n \end{aligned} \tag{21.5.7}$$

As n becomes large, the ratio of the volume of a ball to the volume of the circumscribed (hyper-) cube rapidly becomes small,

$$\frac{V_n}{2^n} \to 0, \qquad n \to \infty \tag{21.5.8}$$

21.5.1 Picking a Random Point on the Sphere

You don't get a random point on the sphere in n dimensions by picking uniformly random values for the $n-1$ angles in equation (21.5.2), just as you don't get a random point on the Earth's surface by throwing darts at a Mercator map (or any other non-equal-area projection).

An elegant general method is to generate n independent, identically distributed, normal (Gaussian) deviates of zero mean, say $y_0, \ldots, y_{n-1}$ (see §7.3), and then calculate a point $\mathbf{x}$ on the unit sphere in n dimensions by

$$\mathbf{x} = \frac{\mathbf{y}}{|\mathbf{y}|} \tag{21.5.9}$$

or, in other words,

$$x_i = y_i \Bigg/ \sqrt{\sum_{j=0}^{n-1} y_j^2} \tag{21.5.10}$$

This works because the spherically symmetric Gaussian distribution in n dimensions trivially factorizes into a product of independent one-dimensional Gaussians. If you want a random point inside the enclosed n-volume, generate an additional *uniform* random deviate u in $[0, 1]$ and calculate the point's coordinates as

$$x_i = u^{1/n}\, y_i \Bigg/ \sqrt{\sum_{j=0}^{n-1} y_j^2} \tag{21.5.11}$$

You can of course scale to any other radius of sphere.

Faster special methods are available for the spheres in two, three, and four dimensions. For two dimensions, the circle, pick u_0 and u_1 uniform in $[-1, 1]$, rejecting choices for which $u_0^2 + u_1^2 > 1$. This picks a random point inside the unit circle. Now scale in the obvious way to get a point on the circle,

$$x_0 = \frac{u_0}{\sqrt{u_0^2 + u_1^2}}, \qquad x_1 = \frac{u_1}{\sqrt{u_0^2 + u_1^2}} \tag{21.5.12}$$

(We already discussed this method in §7.3, under Cauchy deviates.)

A faster method for three dimensions, also using only two random deviates, is due to Marsalgia [1]. Pick a point *inside* the unit circle (u_0, u_1) as above. Then a random point on the sphere in three dimensions is

$$\begin{aligned} x_0 &= 2u_0\sqrt{1 - u_0^2 - u_1^2} \\ x_1 &= 2u_1\sqrt{1 - u_0^2 - u_1^2} \\ x_2 &= 1 - 2(u_0^2 + u_1^2) \end{aligned} \tag{21.5.13}$$

For the sphere in four dimensions, pick two independent points *inside* the unit circle as above, (u_0, u_1) and (u_2, u_3). Then a random point on the sphere in four dimensions is [1]

$$\begin{aligned} x_0 &= u_0 \\ x_1 &= u_1 \\ x_2 &= u_2\sqrt{\frac{1 - u_0^2 - u_1^2}{u_2^2 + u_3^2}} \\ x_3 &= u_3\sqrt{\frac{1 - u_0^2 - u_1^2}{u_2^2 + u_3^2}} \end{aligned} \tag{21.5.14}$$

Unfortunately, there is no known generalization to higher dimensions.

21.5.2 Picking a Random Rotation Matrix

Don't confuse this with picking a point on a sphere. A rotation matrix $\mathbf{M}$ in n dimensions is an orthogonal $n \times n$ matrix. For a *proper rotation*, $\mathbf{M}$ must have determinant 1. The other possibility, determinant -1, represents an *improper rotation*, decomposable into a proper rotation followed by a reflection. The rotation matrix $\mathbf{M}$ maps any point $\mathbf{x}$ to a new point $\mathbf{x}'$ by

$$\mathbf{x}' = \mathbf{M} \cdot \mathbf{x} \tag{21.5.15}$$

A general method for picking a uniformly random rotation matrix is to fill an $n \times n$ matrix $\mathbf{G}$ with independent, identically distributed, normal (Gaussian) deviates of zero mean. Then, use `QRdcmp` in §2.10 to construct the QR decomposition, namely $\mathbf{G} = \mathbf{Q} \cdot \mathbf{R}$. Except for the possibility that it might have the wrong sign of determinant, the matrix $\mathbf{Q}$ is now a uniformly random rotation matrix. The method used in `QRdcmp` is to apply $n - 1$ Householder transformations, each of which is

a reflection with determinant -1. Thus, to get determinant 1, we do nothing to $\mathbf{Q}$ if n is odd; if n is even, we simply interchange any pair of rows in $\mathbf{Q}$, giving the final answer.

For large n the work of doing the decomposition scales as $O(n^3)$, which can be burdensome. For faster, but more complicated, methods, see [2,3].

Faster special methods are available for two and three dimensions. A random two-dimensional rotation matrix has components that are the sine and cosine of a random angle θ in $[0, 2\pi]$,

$$\begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{pmatrix} \tag{21.5.16}$$

We get the components without trigonometric function calls by using (21.5.12) to find a random point on the unit circle and then taking $\cos\theta = x_0$ and $\sin\theta = x_1$.

In the case of three dimensions, a fast method is to use equation (21.5.14) to generate a random point on the sphere in four dimensions, and then to construct the 3×3 orthogonal matrix,

$$\begin{bmatrix} 1-2(x_1^2+x_2^2) & 2(x_0x_1-x_3x_2) & 2(x_0x_2+x_3x_1) \\ 2(x_0x_1+x_3x_2) & 1-2(x_0^2+x_2^2) & 2(x_1x_2-x_3x_0) \\ 2(x_0x_2-x_3x_1) & 2(x_1x_2+x_3x_0) & 1-2(x_0^2+x_1^2) \end{bmatrix} \tag{21.5.17}$$

which will be uniformly random among all rotations [4,5].

CITED REFERENCES AND FURTHER READING:

Marsaglia, G. 1972, "Choosing a Point from the Surface of a Sphere," *Annals of Mathematical Statistics*, vol. 43, pp. 645–646.[1]

Genz, A. 2000, "Methods for Generating Random Orthogonal Matrices," in *Monte Carlo and Quasi-Monte Carlo Methods*, Proceedings of the Third International Conference on Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing (MCQMC98) (Berlin: Springer).[2]

Anderson, T.W., Olkin, I., and Underhill, L.G. 1987, "Generation of Random Orthogonal Matrices," *SIAM Journal on Scientific and Statistical Computing*, vol. 8, pp. 625–629.[3]

Shoemake, K. 1985, "Animating Rotation with Quaternion Curves," *Computer Graphics*, Proceedings of SIGGRAPH 1985, vol. 19, pp. 245–254.[4]

Shoemake, K. 1992, "Uniform Random Rotations," in *Graphics Gems III*, Kirk, D., ed. (Cambridge, MA: Academic Press), pp. 124–132.[5]

21.6 Triangulation and Delaunay Triangulation

We can informally define a *triangulation* of a set of N points in the plane as follows: Connect the given points by straight-line segments as many times as you can without any two segments crossing. When you can't connect any more, you have a triangulation. Obviously there are many triangulations of a given set of points. Figure 21.6.1 shows three triangulations of the same set of 50 points. Two are "random," where the informal definition was pretty much followed literally. The third one is a very special triangulation, called a Delaunay triangulation. In a sense that we will make more precise below, it is the triangulation whose triangles best avoid small angles and long sides.

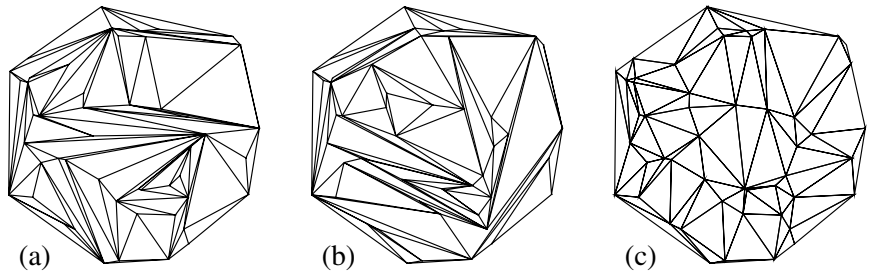


Figure 21.6.1. Three triangulations of the same 50 random points: (a) and (b) are "bad" (random) triangulations, while (c) is a "good" (Delaunay) triangulation. The number of lines and triangles is the same in each case.

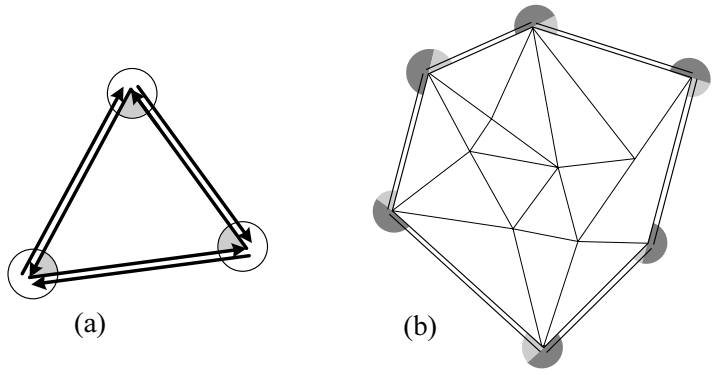


Figure 21.6.2. How to count lines and triangles in a triangulation. (a) Each triangle "uses up" $1/2$ of a point, and $3/2$ lines. (b) The n points in the convex hull "use up" $n/2+1$ points and $n/2$ lines.

All triangulations of a given set of points have the same outer boundary, called the *convex hull* of the point set. This should be evident, again informally, from the definition of triangulation: A line segment (edge) on the outer convex boundary can't interfere with any other actual or potential interior edges, so it will always be added before the stopping rule is reached. The number of points n (and also edges) in the convex hull is at least three, and can be as large as N, e.g., if the points all lie on a circle. (Here and below, we will ignore degenerate cases like "all points lie on a line.")

It is surprising, perhaps, that all triangulations of a given point set have the same number of lines (L) and triangles (T), given explicitly by the relations

$$\begin{aligned} L &= 3N - n - 3 \\ T &= 2N - n - 2 \end{aligned} \tag{21.6.1}$$

The proof, known to Gauss, is very easy if you consult Figure 21.6.2. Since the interior angles of a triangle sum to π radians, each triangle "uses up" half a point's worth of angle. It is useful to think about each line as being two half-lines, representing the two possible directions of traversal in clockwise triangles. Then, each triangle uses up three half-lines. We must separately account for the vertices on the convex hull, as follows: Each such point uses up π radians on its own (dark-shaded angles in the figure), plus (sum of the light-shaded exterior angles) 2π additional radians in going

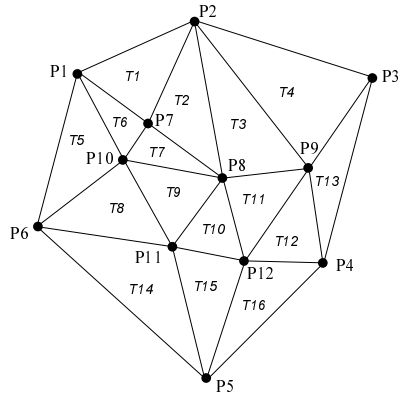


Figure 21.6.3. Example of a triangulation with $N = 12$, $n = 6$, $T = 16$, and $L = 27$, values that satisfy equation (21.6.1).

around the convex hull. These considerations give the relations

$$\begin{aligned} 2\pi N &= \pi T + \pi n + 2\pi \qquad \text{(account for radians)} \\ 2L &= 3T + n \qquad\qquad\quad \text{(account for half-lines)} \end{aligned} \tag{21.6.2}$$

which can be rearranged to give equation (21.6.1). Figure 21.6.3 shows the triangulation of Figure 21.6.2(b) with its points and triangles enumerated.

21.6.1 Delaunay Triangulation

Boris Nikolaevich Delone (1890–1980), a Russian mathematician also celebrated as a rock climber, first published the ideas behind Delaunay triangulation in 1934. Since his paper was written in French, his name was transliterated so as to be pronounced (approximately) correctly by French speakers. A better English pronunciation might be "dyeh-LOAN-yeh."

Delaunay triangulations have a number of remarkable properties and can be defined in various abstract ways. However, we'll take as the definition one very concrete property, shown in Figure 21.6.4. Consider all triangulations in which four points, A, B, C, D, are the vertices of two back-to-back triangles. Then, one can get a different triangulation by deleting the common edge (BD in the figure) and replacing it by the other diagonal of the quadrilateral (AC in the figure). A Delaunay triangulation is defined as one that always chooses the diagonal that gives the *largest minimum angle* for the six interior angles in the two triangles. The edge shown as BD is thus *illegal* for a Delaunay triangulation, while AC is termed *legal*. Changing a triangulation from an illegal edge to a legal one is called an *edge flip*. When any two triangles have a common edge, exactly one configuration, unchanged or edge-flipped, is legal (unless all four points lie on a circle, in which case both are legal).

This "largest minimum angle" property is geometrically equivalent to other statements about the points A, B, C, D. One such statement is that the circumcircle of an illegal triangle, like ABD or BCD in part (a) of the figure, always contains another point, C or A, respectively. For a legal triangle, as in part (b) of the figure, this is never the case. One can use this as a starting point to prove the theorem:

- The circumcircle of any triangle in a Delaunay triangulation contains no other vertices.

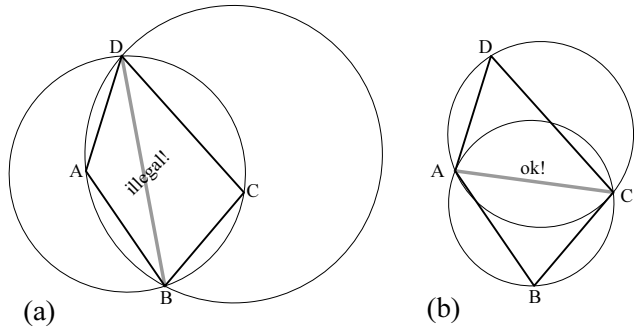


Figure 21.6.4. A Delaunay triangulation can be defined as one in which back-to-back triangles always have a larger minimum angle than they would have if their common edge were flipped to the other diagonal. Equivalently, any triangle's circumcircle does not contain another vertex. A so-called *edge flip* converts (a) to (b) in the figure.

Although the largest minimum angle property was defined locally, for one quadrilateral at a time, it can be shown to imply a remarkable global theorem:

- Among all triangulations of a set of points, a Delaunay triangulation has the largest minimum angles, as defined by sorting all angles from smallest to largest and comparing lexicographically to the angles of any other triangulation.

Comparing lexicographically means: First compare the smallest angle; if there is a winner, stop. If there is a tie on the smallest angle, compare the second-smallest angle. And so on.

Another theorem is

- Two vertices are connected by a Delaunay edge if and only if there is some circle that contains them and contains no other vertices.

If the points in a set have generic positions, meaning that no three are collinear and no four lie on the same circle, then the Delaunay triangulation exists and is unique; any method for constructing it will give the identical set of triangles.

You might wonder whether a Delaunay triangulation is also a *minimum weight* triangulation, defined as the triangulation with the smallest total of edge lengths. The answer is, in general, no. While minimum weight triangulations might be useful in applications, it is not even known whether they can be constructed in less than time that grows exponentially with N. Delaunay construction, on the other hand, is fast, $O(N \log N)$. So, in practice, Delaunay is what we've got!

So, how do we construct a Delaunay triangulation? Conceptually, we can start with *any* triangulation and then eliminate illegal edges, by edge flips, as long as possible. This must terminate in a Delaunay triangulation after a finite number of flips because (i) each flip changes and increases the lexicographic order in the list of angles, and (ii) there are only a finite number of possible triangulations. Although, as stated, this is not an efficient algorithm, it can be readily be turned into one, the so-called *randomized incremental algorithm* [1].

This algorithm, which we now implement, is "incremental" in that it adds points to the triangulation one at a time, maintaining a Delaunay triangulation at each stage. It is "randomized" in that the points are added in a random order. It turns out that the randomization (almost) guarantees $O(N \log N)$ expected time for the algorithm. (Without randomization, one could encounter pathological cases with

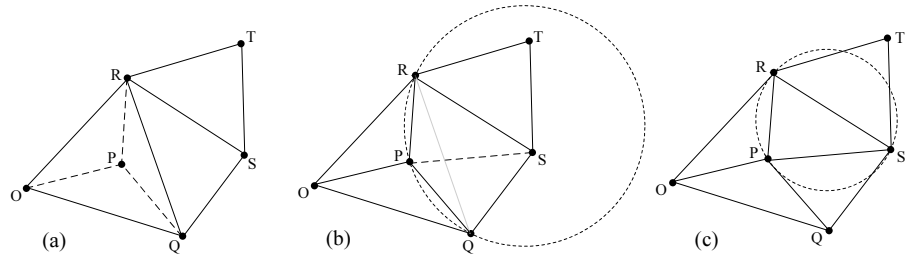


Figure 21.6.5. Steps in inserting a new point into a Delaunay triangulation. (a) Connect new point P to vertices of enclosing triangle. (b) Check enclosing triangle for illegal edges (here, replace QR by PS). (c) Recursively check any new triangles created that have P as a vertex. (Here, RS is legal, so we terminate.)

$O(N^2)$ running time.)

Figure 21.6.5 shows the procedure for adding a new point P that lies within an existing triangle. First, connect it to the vertices of the enclosing triangle. This creates three new triangles. (We exclude the special case where P is exactly on an existing line. More on this below.) Next, check whether the edges opposite P in the three new triangles are legal or illegal. If they are illegal, do edge flips. Each edge flip creates two new triangles with P as a vertex, and (therefore) with two edges opposite P that now also need to be checked for legality. So the process is recursive, but it never wanders away from P. That is the key point: The only edges that can be made illegal by inserting point a point P are edges opposite P in triangles that include P. The proof that the algorithm is $O(N \log N)$ uses this fact, and then bounds the average number of triangles of which P can be a vertex by relations like those of equation (21.6.1). (For details in the proof, see [2].)

Since, up to now, we only know how to add a point P that falls inside the triangulation, how can we get started? An easy way is to add three "fictitious" points to the set of points, forming a very large starting triangle that will enclose all the "real" points subsequently added. Then, at the very end, the fictitious points and all edges connecting to them are removed. Strictly speaking, the fictitious points must actually be treated as if at infinity (thus requiring special logic in the code whenever they are referenced). If their distance is merely finite, the constructed triangulation may be "not-quite" Delaunay. For example, its outer boundary (convex hull) could in unusual cases be left slightly concave, with small negative angles on the order of the diameter of the "real" point set divided by the distance to the "fictitious" points.

That's enough general information. We next get into the details.

21.6.2 Implementation Details

Since most readers skip a section with this title, this is a good place for us to confess a couple of dirty tricks in our Delaunay implementation, whose purpose is to keep the code and its explication to manageable length. If you need a bullet-proof Delaunay code, with no such tricks, a Web search will turn up several that are freely available. Our code is short and fast, and fine for its intended purpose; but it is approximate in two respects: First, we don't take the initial bounding triangle off to infinity (as we sanctimoniously advised, above). Instead, it lies at a distance of about `bigscale` (an adjustable parameter, default value 1000), measured in units of the bounding-box size of the set of points. Second, we don't provide for the special case, mentioned above, where the point being added falls on an existing edge (or does so within roundoff tolerance). For generic point locations, this should "never" happen; but in real

life it "always" happens, because users love to try test examples with points in regular patterns! When we detect this problem, we randomly alter the location of the offending point by a small fraction `fuzz` (another adjustable parameter, default 10^{-6}) of the bounding box dimensions.

A very important implementation detail, not yet discussed, is how to find in which existing triangle a new point lies. Conceptually, we might throw the triangles into a QO tree (§21.8), but this would not yield the desired $O(N \log N)$ behavior for our algorithm. A better solution, well established in the literature, is to maintain a tree structure of the descendants of any given triangle that ever existed in the construction. That is, starting with the huge "root" triangle, whenever a triangle is subdivided into three new triangles, we set pointers to its three daughters. And, when two triangles are lost in an edge flip, and two new triangles are created, we make the new triangles daughters of *both* of the lost triangles (even though each lost triangle contains only a part of each new one). In this scheme a triangle has two or three daughters at most, so we can easily reserve space for the pointers explicitly (i.e., no expandable linked lists needed).

With this structure, it is very fast to locate a point within the existing triangulation: Start at the root triangle, and recursively pick whichever daughter contains the point. When you reach a terminal node in the tree, you will have found a triangle in the current triangulation that contains the point. We thus need a structure for a "triangle element" or `Triel`:

delaunay.h

```
struct Triel {
```
Structure for an element in a descendancy tree of triangles, each having at most three daughters.
```
    Point<2> *pts;                          Pointer to the array of the points.
    Int p[3];                               The triangle's three vertices, always in CCW order.
    Int d[3];                               Pointers for up to three daughters.
    Int stat;                               Nonzero if this element is "live."
    void setme(Int a, Int b, Int c, Point<2> *ptss) {
```
        Set the data in a `Triel`.
```
        pts = ptss;
        p[0] = a; p[1] = b; p[2] = c;
        d[0] = d[1] = d[2] = -1;            The values −1 mean no daughters.
        stat = 1;                           Create as "live."
    }
    Int contains(Point<2> point) {
```
        Return 1 if `point` is in the triangle, 0 if on boundary, −1 if outside. (CCW triangle is assumed.)
```
        Doub d;
        Int i,j,ztest = 0;
        for (i=0; i<3; i++) {
            j = (i+1) %3;
            d = (pts[p[j]].x[0]-pts[p[i]].x[0])*(point.x[1]-pts[p[i]].x[1]) -
                (pts[p[j]].x[1]-pts[p[i]].x[1])*(point.x[0]-pts[p[i]].x[0]);
            if (d < 0.0) return -1;
            if (d == 0.0) ztest = 1;
        }
        return (ztest? 0 : 1);
    }
};
```

We create an big enough array of `Triel`s at the start, and use integers to point to array elements. We've omitted any explicit constructor or assignment operators in `Triel`, since they are not needed for our use here. Be sure to add them if you use `Triel` in any other way.

We will need a way to do two other fast lookups: (1) Given a point and opposite edge in a triangle, find the fourth point in the quadrilateral, that is, the point (if any) on the other side of the given edge. (2) Given three points, find the index of their triangle (if it exists) in an array of `Triel` elements. Our strategy is to use hash memories (respectively called `linehash` and `trihash`) for these two functions. In particular, whenever we create a triangle (always CCW) with vertices A, B, C, we store an index pointing to each point under a specially constructed key,

$$\texttt{linehash}(h(B) - h(C)) \leftarrow A \qquad \text{(et cyc.)} \tag{21.6.3}$$

where the function h is a 64-bit hash function, and "et cyc." means do the same thing for the

other two cyclic permutations of A, B, C. The trick here is that, if we ever want to find the point on the other side of edge BC, we just look for a key $h(C) - h(B)$ ("negative" of the key in equation 21.6.3) in the hash table. The similar trick for storing and retrieving `Triel`s is that, when we create a `Triel` at location j in the storage array, we set

$$\texttt{trihash}(h(A) \ \hat{} \ h(B) \ \hat{} \ h(C)) \leftarrow j \tag{21.6.4}$$

where $\hat{}$ is the XOR operation. Since this key is symmetric in A, B, C, we can find a triangle knowing its vertices in any order.

Since we are computing hash keys "by hand," we can signal the two hash memories to use a null (therefore fast) hash of their own. That, and a utility for determining whether a point is inside the circumcircle of three other points, are the following two code fragments:

delaunay.h
```
Doub incircle(Point<2> d, Point<2> a, Point<2> b, Point<2> c) {
```
Return positive, zero, or negative value if point `d` is respectively inside, on, or outside the circle through points `a`, `b`, and `c`.
```
    Circle cc = circumcircle(a,b,c);        Routine defined in §21.3.
    Doub radd = SQR(d.x[0]-cc.center.x[0]) + SQR(d.x[1]-cc.center.x[1]);
    return (SQR(cc.radius) - radd);
}

struct Nullhash {
```
Null hash function. Use a key (assumed to be already hashed) as its own hash.
```
    Nullhash(Int nn) {}
    inline Ullong fn(const void *key) const { return *((Ullong *)key); }
};
```

These are all the preliminaries we need before declaring the `Delaunay` structure.

delaunay.h
```
struct Delaunay {
```
Structure for constructing a Delaunay triangulation from a given set of points.
```
    Int npts,ntri,ntree,ntreemax,opt;       Number of points, triangles, elements in the
    Doub delx,dely;                             Triel list, and maximum of same. Size
    vector< Point<2> > pts;                     of the bounding box.
    vector<Triel> thelist;                  The list of Triel elements.
    Hash<Ullong,Int,Nullhash> *linehash;    Create the hash memories with null hash
    Hash<Ullong,Int,Nullhash> *trihash;         function.
    Int *perm;
    Delaunay(vector<Point<2> > &pvec, Int options = 0);
    Construct the Delaunay triangulation from a vector of points. The variable options is used
    by some applications.
    Ranhash hashfn;                         The raw hash function.
    Doub interpolate(const Point<2> &p, const vector<Doub> &fnvals,
        Doub defaultval=0.0);
    The next four functions are explained in detail below.
    void insertapoint(Int r);
    Int whichcontainspt(const Point<2> &p, Int strict = 0);
    Int storetriangle(Int a, Int b, Int c);
    void erasetriangle(Int a, Int b, Int c, Int d0, Int d1, Int d2);
    static Uint jran;                       Random number counter.
    static const Doub fuzz, bigscale;
};
const Doub Delaunay::fuzz  = 1.0e-6;        Adjust if you wish. See text.
const Doub Delaunay::bigscale = 1000.0;     Adjust if you wish. See text.
Uint Delaunay::jran = 14921620;
```

The variable `jran` is used in conjunction with the hash function as a convenient random number generator. The function `interpolate()` is for the application of interpolating a function on an irregular mesh, to be discussed in §21.7. Everything else should become clear as we proceed.

The action starts with the constructor. We compute a bounding box for the set of points, construct and store the "huge" root triangle enclosing the points, create a random permuta-

tion to be the order in which points are added, and then (for the real work) call the function `insertapoint()` for each point in turn. After that there is just some cleanup housekeeping.

delaunay.h

```
Delaunay::Delaunay(vector< Point<2> > &pvec, Int options) :
    npts(pvec.size()), ntri(0), ntree(0), ntreemax(10*npts+1000),
    opt(options), pts(npts+3), thelist(ntreemax) {
```

Construct Delaunay triangulation from a vector of points pvec. If bit 0 in options is nonzero, hash memories used in the construction are deleted. (Some applications may want to use them and will set options to 1.)

```
    Int j;
    Doub xl,xh,yl,yh;
    linehash = new Hash<Ullong,Int,Nullhash>(6*npts+12,6*npts+12);
    trihash = new Hash<Ullong,Int,Nullhash>(2*npts+6,2*npts+6);
    perm = new Int[npts];                Permutation for randomizing point order.
    xl = xh = pvec[0].x[0];              Copy points to local store and calculate their
    yl = yh = pvec[0].x[1];                  bounding box.
    for (j=0; j<npts; j++) {
        pts[j] = pvec[j];
        perm[j] = j;
        if (pvec[j].x[0] < xl) xl = pvec[j].x[0];
        if (pvec[j].x[0] > xh) xh = pvec[j].x[0];
        if (pvec[j].x[1] < yl) yl = pvec[j].x[1];
        if (pvec[j].x[1] > yh) yh = pvec[j].x[1];
    }
    delx = xh - xl;                      Store bounding box dimensions, then construct
    dely = yh - yl;                          the three fictitious points and store them.
    pts[npts] = Point<2>(0.5*(xl + xh), yh + bigscale*dely);
    pts[npts+1] = Point<2>(xl - 0.5*bigscale*delx,yl - 0.5*bigscale*dely);
    pts[npts+2] = Point<2>(xh + 0.5*bigscale*delx,yl - 0.5*bigscale*dely);
    storetriangle(npts,npts+1,npts+2);
    Create a random permutation:
    for (j=npts; j>0; j--) SWAP(perm[j-1],perm[hashfn.int64(jran++) % j]);
    for (j=0; j<npts; j++) insertapoint(perm[j]);     All the action is here!
    for (j=0; j<ntree; j++) {            Delete the huge root triangle and all of its con-
      if (thelist[j].stat > 0) {             necting edges.
          if (thelist[j].p[0] >= npts || thelist[j].p[1] >= npts ||
          thelist[j].p[2] >= npts) {
              thelist[j].stat = -1;
              ntri--;
          }
      }
    }
    if (!(opt & 1)) {                    Clean up, unless option bit says not to.
        delete [] perm;
        delete trihash;
        delete linehash;
    }
}
```

The guts of the algorithm as previously described are in `insertapoint()`. We first locate the triangle that contains the new point. (Failure here can mean only that the point lies on an existing line, in which case we fuzz the point's location, as we confessed above, and try again.) We store three new triangles and delete the old one. Then, we locate and fix any illegal edges, doing the recursion by a simple last-in-first-out stack of pending edges to check.

delaunay.h

```
void Delaunay::insertapoint(Int r) {
```

Add the point with index r incrementally to the Delaunay triangulation.

```
    Int i,j,k,l,s,tno,ntask,d0,d1,d2;
    Ullong key;
    Int tasks[50], taski[50], taskj[50];     Stacks (3 vertices) for legalizing edges.
    for (j=0; j<3; j++) {                    Find triangle containing point. Fuzz if it
        tno = whichcontainspt(pts[r],1);         lies on an edge.
        if (tno >= 0) break;                 The desired result: Point is OK.
```

```
            pts[r].x[0] += fuzz * delx * (hashfn.doub(jran++)-0.5);
            pts[r].x[1] += fuzz * dely * (hashfn.doub(jran++)-0.5);
        }
        if (j == 3) throw("points degenerate even after fuzzing");
        ntask = 0;
        i = thelist[tno].p[0]; j = thelist[tno].p[1]; k = thelist[tno].p[2];
```

The following line is relevant only when the indicated bit in opt is set. This feature is used by the convex hull application and causes any points already known to be interior to the convex hull to be omitted from the triangulation, saving time (but giving in an incomplete triangulation).

```
        if (opt & 2 && i < npts && j < npts && k < npts) return;
        d0 =storetriangle(r,i,j);                  Create three triangles and queue them
        tasks[++ntask] = r; taski[ntask] = i; taskj[ntask] = j;     for legal edge tests.
        d1 = storetriangle(r,j,k);
        tasks[++ntask] = r; taski[ntask] = j; taskj[ntask] = k;
        d2 = storetriangle(r,k,i);
        tasks[++ntask] = r; taski[ntask] = k; taskj[ntask] = i;
        erasetriangle(i,j,k,d0,d1,d2);             Erase the old triangle.
        while (ntask) {                            Legalize edges recursively.
            s=tasks[ntask]; i=taski[ntask]; j=taskj[ntask--];
            key = hashfn.int64(j) - hashfn.int64(i);      Look up fourth point.
            if ( ! linehash->get(key,l) ) continue;       Case of no triangle on other side.
            if (incircle(pts[l],pts[j],pts[s],pts[i]) > 0.0){    Needs legalizing?
                d0 = storetriangle(s,l,j);                Create two new triangles
                d1 = storetriangle(s,i,l);
                erasetriangle(s,i,j,d0,d1,-1);            and erase old ones.
                erasetriangle(l,j,i,d0,d1,-1);
                key = hashfn.int64(i)-hashfn.int64(j);    Erase line in both directions.
                linehash->erase(key);
                key = 0 - key;                            Unsigned, hence binary minus.
                linehash->erase(key);
                Two new edges now need checking:
                tasks[++ntask] = s; taski[ntask] = l; taskj[ntask] = j;
                tasks[++ntask] = s; taski[ntask] = i; taskj[ntask] = l;
            }
        }
    }
```

The only pieces left are the utility functions for finding the triangle that contains a point, and for storing and erasing a triangle. When we "erase" a triangle, we actually only mark it as inactive in the current triangulation, and we set its daughters in the descendancy tree, as already discussed.

```
Int Delaunay::whichcontainspt(const Point<2> &p, Int strict) {                    delaunay.h
```

Given point p, return index in `thelist` of the triangle in the triangulation that contains it, or return -1 for failure. If `strict` is nonzero, require strict containment, otherwise allow the point to lie on an edge.

```
    Int i,j,k=0;
    while (thelist[k].stat <= 0) {          Descend in tree until reach a "live" triangle.
        for (i=0; i<3; i++) {               Check up to three daughters.
            if ((j = thelist[k].d[i]) < 0) continue;      Daughter doesn't exist.
            if (strict) {
                if (thelist[j].contains(p) > 0) break;
            } else {                        Yes, descend on this branch.
                if (thelist[j].contains(p) >= 0) break;
            }
        }
        if (i == 3) return -1;              No daughters contain the point.
        k = j;                              Set new mother.
    }
    return k;                               Normal return.
}
```

```
void Delaunay::erasetriangle(Int a, Int b, Int c, Int d0, Int d1, Int d2) {
```
Erase triangle abc in trihash and inactivate it in thelist after setting its daughters.
```
    Ullong key;
    Int j;
    key = hashfn.int64(a) ^ hashfn.int64(b) ^ hashfn.int64(c);
    if (trihash->get(key,j) == 0) throw("nonexistent triangle");
    trihash->erase(key);
    thelist[j].d[0] = d0; thelist[j].d[1] = d1; thelist[j].d[2] = d2;
    thelist[j].stat = 0;
    ntri--;
}

Int Delaunay::storetriangle(Int a, Int b, Int c) {
```
Store a triangle with vertices a, b, c in trihash. Store its points in linehash under keys to opposite sides. Add it to thelist, returning its index there.
```
    Ullong key;
    thelist[ntree].setme(a,b,c,&pts[0]);
    key = hashfn.int64(a) ^ hashfn.int64(b) ^ hashfn.int64(c);
    trihash->set(key,ntree);
    key = hashfn.int64(b)-hashfn.int64(c);
    linehash->set(key,a);
    key = hashfn.int64(c)-hashfn.int64(a);
    linehash->set(key,b);
    key = hashfn.int64(a)-hashfn.int64(b);
    linehash->set(key,c);
    if (++ntree == ntreemax) throw("thelist is sized too small");
    ntri++;
    return (ntree-1);
}
```

You might wonder how to get the answer *out* of our `Delaunay` structure. We have not provided a function for this, because it so much depends on what you want to do with the answer. The general idea, however, is that you just loop through `thelist`$[j]$ for $0 \le j <$ `nlist`. Each element is a `Triel`. If its value of `save` is ≤ 0, ignore it and go on. If it is 1, then the element represents a triangle in the final Delaunay triangulation. There should be `ntri` of these elements in all. The element's array `p[]` has integers that point to the triangle's three points in your vector of points. Several routines in the next section mine the `Delaunay` structure for points, edges, or triangles and can be used as template examples.

Figure 21.6.6 shows sample output for a Delaunay triangulation of 300 points.

CITED REFERENCES AND FURTHER READING:

Guibas, L.J., Knuth, D.E., and Sharir, M. 1992, "Randomized Incremental Construction of Delaunay and Voronoi Diagrams," *Algorithmica*, vol. 7, pp. 381–413.[1]

Lischinski, D. 1994, "Incremental Delaunay Triangulation," in *Graphics Gems IV*, Heckbert, P.S., ed. (Cambridge, MA: Academic Press). [Shows use of linked data structure instead of our use of hash memory.]

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 9.[2]

O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press), §5.3.

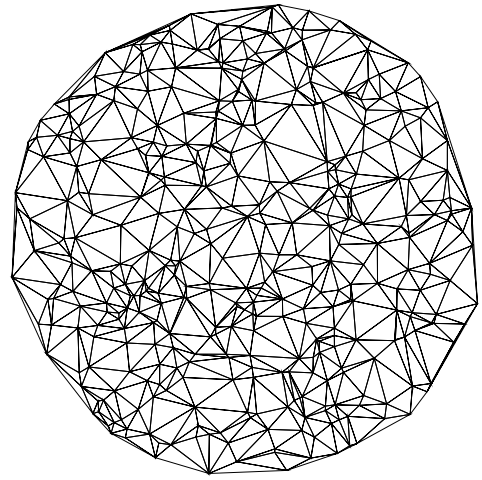


Figure 21.6.6. Delaunay triangulation of 300 points randomly chosen within a circle, as computed by the routines in this section.

21.7 Applications of Delaunay Triangulation

Emerging from the thicket of detail that was needed to implement the Delaunay triangulation, we are now ready to make use of it in several important applications. In this section we assume that you have a vector of points (say, `vecp`), and that you have invoked the code in §21.6 to construct a `Delaunay` structure. This usually means writing just the one line of code,

```
Delaunay mygrid(vecp);
```

So, what next?

21.7.1 Two-Dimensional Interpolation on an Irregular Grid

This is probably the most-asked-for algorithm that was missing from the previous two editions of *Numerical Recipes*. The basic setup is very simple. You are given a set of N points in the plane. You triangulate that set with a "good" triangulation, that is, one favoring short lines and big angles — in other words, Delaunay. You evaluate a function of interest at each of the points, and store the values in a vector (in the same order as the vector of points, of course).

Now it is easy to interpolate the function at a new point $\mathbf{p}$ that lies within the triangulation, that is, specifically, within the convex hull of your set of points. First, locate which triangle the point falls in. This takes only $O(\log N)$ operations if you use the `whichcontainspt()` method of the `Delaunay` structure. Then, linearly interpolate between the three function values at the three triangle vertices. The linear interpolation is uniquely defined, because (imagining your function plotted in the third dimension above the plane in which $\mathbf{p}$ lies) three points uniquely define a plane in three dimensions .

Constructively, the linear interpolation is easily done using barycentric coordinates as defined in equation (21.3.10), which in turn reduces to using the triangle area formula (equation 21.3.1) three times. Appropriately normalized, each barytropic coordinate value is exactly the weight of its corresponding vertex.

These ideas are implemented in the following function:

delaunay.h

```
Doub Delaunay::interpolate(const Point<2> &p,
const vector<Doub> &fnvals, Doub defaultval) {
```

Triangular grid interpolation of a function. Given an arbitrary point p and a vector of function values fnvals at the points that were used to construct the Delaunay structure, return the linearly interpolated function value in the triangle in which p lies. If p lies outside of the triangulation, instead return defaultval.

```
    Int n,i,j,k;
    Doub wgts[3];
    Int ipts[3];
    Doub sum, ans = 0.0;
    n = whichcontainspt(p);                    Locate the point in the triangulation.
    if (n < 0) return defaultval;              Point outside of convex hull.
    for (i=0; i<3; i++) ipts[i] = thelist[n].p[i];
    for (i=0,j=1,k=2; i<3; i++,j++,k++) {      Calculate the barycentric coordinates, pro-
        if (j == 3) j=0;                           portional to the weights.
        if (k == 3) k=0;
        wgts[k]=(pts[ipts[j]].x[0]-pts[ipts[i]].x[0])*(p.x[1]-pts[ipts[i]].x[1])
            - (pts[ipts[j]].x[1]-pts[ipts[i]].x[1])*(p.x[0]-pts[ipts[i]].x[0]);
    }
    sum = wgts[0] + wgts[1] + wgts[2];         Normalization of the weights.
    if (sum == 0) throw("degenerate triangle");
    for (i=0; i<3; i++) ans += wgts[i]*fnvals[ipts[i]]/sum;     Linear interpolation.
    return ans;
}
```

Keep in mind that you should not expect high accuracy from linear interpolation. The interpolated function is piecewise linear, and continuous within the convex hull, but it has discontinuous derivatives in the direction perpendicular to the triangle's edges. On a triangle edge, it interpolates between the two function values at each end of the edge. You need a *lot* of triangles to get a reasonable representation of any function with much detailed structure.

21.7.2 Voronoi Diagrams

Around 1907, the Ukrainian mathematician Georgy Fedoseevich Voronoi revisited a problem that had been previously discussed by Dirichlet as early as 1850: Given N points, or *sites*, in the plane, each site $\mathbf{p}$ defines a region that is closer to $\mathbf{p}$ than to any of the other $N-1$ sites. That region is called $\mathbf{p}$'s *Voronoi region*. What are the properties of the Voronoi regions, and how can we construct them?

If you imagine that everyone in a city shops at the closest supermarket ("as the crow flies"), then the Voronoi regions map out the districts served by each supermarket. If you imagine that fires are simultaneously set at sites in a forest, and that they spread circularly at a fixed speed, then the Voronoi regions are the areas burned by each different fire.

Figure 21.7.1, an example of a *Voronoi diagram*, shows the Voronoi regions around 40 sites chosen randomly in the plane. Yes, the boundaries of the Voronoi regions are polygons, although possibly open and extending to infinity. It is obvious, in fact, that the boundary of a site $\mathbf{p}$'s Voronoi region must consist of line segments each lying on the perpendicular bisector of the line connecting $\mathbf{p}$ to some other site, say $\mathbf{q}_i$. That is because the perpendicular bisector is the locus of points equidistant from $\mathbf{p}$ and $\mathbf{q}_i$. So the real questions are, for a given $\mathbf{p}$, which are the $\mathbf{q}_i$'s that contribute boundary segments? and is there a fast way to compute their intersections (the *vertices* of the Voronoi diagram)?

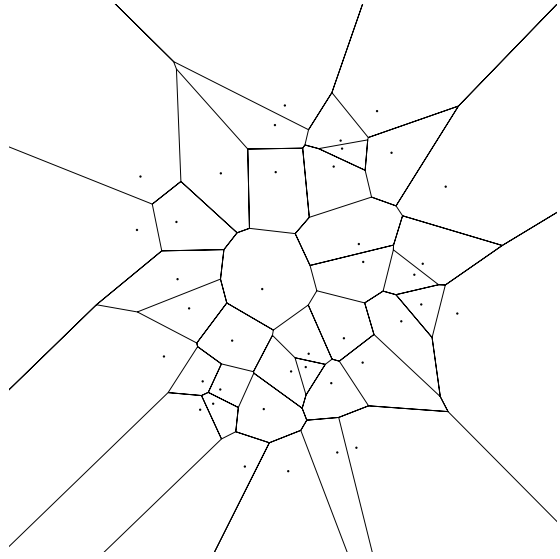


Figure 21.7.1. Voronoi diagram for 40 random sites. Each site has a Voronoi region, the area closer to it than to any other site. The boundaries of the Voronoi regions are straight-line segments that lie on the perpendicular bisectors between pairs of sites.

Remarkably, these questions are completely answered by the Delaunay triangulation of the Voronoi sites. (In fact, many texts start with the Voronoi diagram as fundamental, and then consider Delaunay triangulation as an application. We find it easier to go the other way.)

Some facts are

- Every edge in a site **p**'s Voronoi region boundary lies on the perpendicular bisector of a Delaunay edge that connects to **p**.
- In fact, every Delaunay edge corresponds to exactly one Voronoi edge, and vice versa.
- The vertices of the Voronoi diagram are exactly the circumcenters of the Delaunay triangles.
- The Voronoi diagram and the Delaunay triangulation are *dual graphs* (but don't worry if you don't know what this means)

Figure 21.7.2 shows the key ideas in the proof of the first two facts above. We already know that the boundary is made of *some* perpendicular bisector segments. We need to show that (i) every one of a point's Delaunay edges *does* contribute a segment, and (ii) lines drawn from that point to any other sites *don't* contribute any segments.

Part (a) of the figure shows a piece of Delaunay triangulation around site O. The perpendicular bisectors of OA and OC meet at the point X, which is therefore the center of the circle containing A, O, and C. The issue is whether the Delaunay edge OB can be "blocked" by the other two edges. Now, B must lie inside the circumcircle just mentioned; otherwise, the edge OB would have been an illegal edge when the Delaunay triangulation was constructed. But this means that the perpendicular bisector of OB, labeled UV, must "cut off the corner" at X. Thus it does contribute a segment to the boundary.

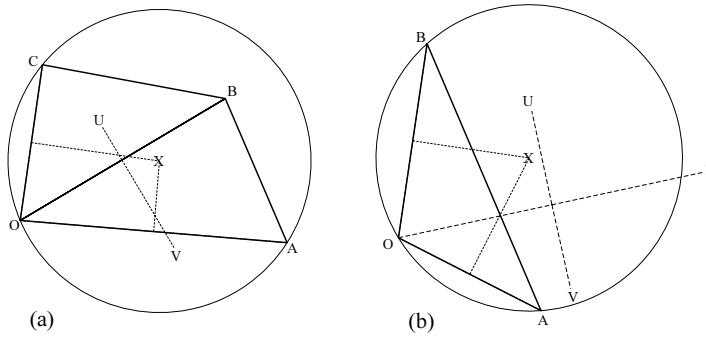


Figure 21.7.2. Key ideas in the proof that every Delaunay edge contributes exactly one Voronoi edge, lying on its perpendicular bisector. (a) Delaunay requires B to lie inside the circle AOC, hence its bisector must clip the corner inside X. (b) Delaunay requires any other site P to lie outside of the circle AOB, hence its bisector can't clip the corner inside X.

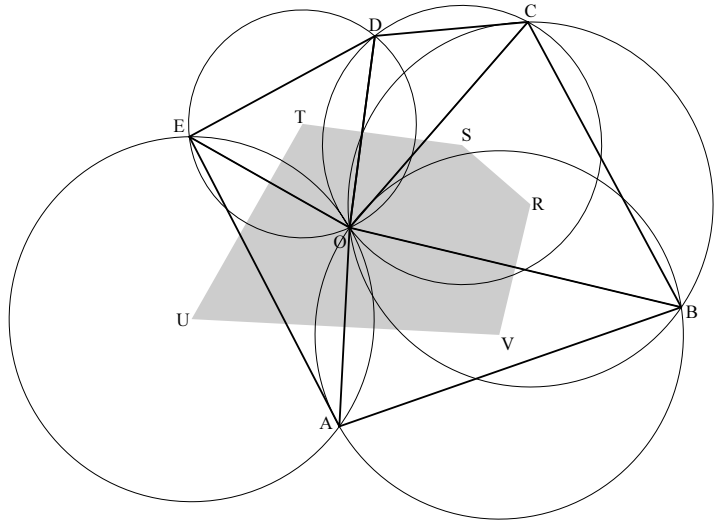


Figure 21.7.3. The circumcenters of the Delaunay triangles around a point O are the vertices of O's Voronoi region (shown as shaded), because the perpendicular bisectors of the Delaunay edges meet at these circumcenters. Notice that a Voronoi edge need not actually intersect the Delaunay edge with which it is associated, as SR and OC.

Part (b) of the figure shows a Delaunay triangle OAB whose edges OA and OB are contributing perpendicular bisector segments to the Voronoi boundary around O. Point P is some other site. Can it somehow worm its way in close enough to contribute a segment of its own bisector, between the other two? Evidently not: We know that the circumcircle of any Delaunay triangle contains no other sites. Since P must lie outside the circumcircle, its bisector UV can't cut off the corner at X.

The fact that the Voronoi vertices are the circumcenters of Delaunay triangles is an immediate consequence of the previous discussion (see Figure 21.7.3). The circumcenters are where the perpendicular bisectors of the edges meet. Since every Delaunay edge contributes a segment, every such circumcenter must be a vertex. Notice, however, that not every Delaunay triangle contains its own circumcenter (as OCD in the figure), so that a segment on the boundary of a Voronoi region need not

actually intersect the Delaunay edge with which it is associated (as RS and OC in the figure).

We can count the number of edges and vertices in a Voronoi diagram with N sites (n of which are on the convex hull) simply by knowing that its dual is the Delaunay triangulation and using equation (21.6.1). The number of Voronoi edges is thus L in that equation, while the number of Voronoi vertices is T. The number of Voronoi regions is by definition N. The unbounded Voronoi regions are exactly those whose points lie on the convex hull of the sites, so there are n of these. It turns out (not immediately obviously) that the average number of edges in one Voronoi region (averaged over all the sites) does not exceed six.

Turning to the implementing code, it is convenient to have a structure for holding Voronoi edges, and also their association with the site that they surround (as an integer pointer to a list of sites).

voronoi.h

```
struct Voredge {
```

Structure for an edge in a Voronoi diagram, containing its two endpoints and an integer pointer to the site of which it is a boundary.

```
    Point<2> p[2];
    Int nearpt;
    Voredge() {}
    Voredge(Point<2> pa, Point<2> pb, Int np) : nearpt(np) {
        p[0] = pa; p[1] = pb;
    }
};
```

Now it is straightforward to define a `Voronoi` structure, as a derived class of the `Delaunay` structure. The constructor creates a Delaunay triangulation of the sites, and then loops over the sites. For each, it first finds any one triangle with the site as a vertex, and then works its way around the site in a circle, navigating counterclockwise from one triangle to the next by looking up their common edge in the `linehash` hash memory. Each triangle's circumcenter is a Voronoi vertex, and a Voronoi edge is stored for each two consecutive circumcenters as the site is circumnavigated.

voronoi.h

```
struct Voronoi : Delaunay {
```

Structure for creating a Voronoi diagram, derived from the `Delaunay` structure.

```
    Int nseg;                                   Number of edges in the diagram.
    VecInt trindx;                              Will index triangles.
    vector<Voredge> segs;                       Will be array of all segments.
    Voronoi(vector< Point<2> > pvec);           Construct Voronoi diagram from array of points.
};

Voronoi::Voronoi(vector< Point<2> > pvec) :
    Delaunay(pvec,1), nseg(0), trindx(npts), segs(6*npts+12) {
```

Constructor for Voronoi diagram of a vector of sites `pvec`. Bit "1" sent to the `Delaunay` constructor tells it not to delete `linehash`.

```
    Int i,j,k,p,jfirst;
    Ullong key;
    Triel tt;
    Point<2> cc, ccp;                           Create a table so that, given a point number, we can find one triangle with it as a vertex.
    for (j=0; j<ntree; j++) {
        if (thelist[j].stat <= 0) continue;
        tt = thelist[j];
        for (k=0; k<3; k++) trindx[tt.p[k]] = j;
    }                                           Now loop over the sites.
    for (p=0; p<npts; p++) {
```

```
            tt = thelist[trindx[p]];
            if (tt.p[0] == p) {i = tt.p[1]; j = tt.p[2];}        Get the vertices into canon-
            else if (tt.p[1] == p) {i = tt.p[2]; j = tt.p[0];}       ical order.
            else if (tt.p[2] == p) {i = tt.p[0]; j = tt.p[1];}
            else throw("triangle should contain p");
            jfirst = j;                          Save starting vertex and its circumcircle.
            ccp = circumcircle(pts[p],pts[i],pts[j]).center;
            while (1) {              Go around CCW, find circumcenters and store segments.
                key = hashfn.int64(i) - hashfn.int64(p);
                if ( ! linehash->get(key,k) ) throw("Delaunay is incomplete");
                cc = circumcircle(pts[p],pts[k],pts[i]).center;
                segs[nseg++] = Voredge(ccp,cc,p);
                if (k == jfirst) break;          Circumnavigation completed.  Normal way out.
                ccp = cc;
                j=i;
                i=k;
            }
        }
    }
```

The result of the `Voronoi` construct is available by looping through the `segs` array from 0 to `nseg-1`. Each array element is a `Voredge` that stores the endpoints, and also the site number with which it is associated. Note that each segment appears twice in the list, with opposite sense, as it is associated in turn with the sites on its two sides.

If you read our confession about dirty tricks in the previous section, you'll want to keep in mind that the "open" Voronoi polygons are actually closed by segments that lie at a distance of order `bigscale` times the size of the bounding box of the sites. Those segments are included in `segs` but appear only once, since there is no site on their other side.

21.7.3 Other Applications

Nearest Neighbors, Again. A line segment that connects a point to its nearest neighbor among a set of points will be an edge in the set's Delaunay triangulation. Informal proof: The nearest neighbor obviously must contribute a boundary to the Voronoi diagram. Formal proof (using a theorem mentioned above): The circle whose diameter connects a point to its nearest neighbor can't contain any other points (they'd be closer than the nearest neighbor), so that diameter must be a Delaunay edge.

Since we can construct the Delaunay triangulation in $O(N \log N)$ time, it follows that we can use it to find all nearest neighbors of a set of N points in $O(N \log N)$ time. The process is as follows: (i) Construct `Delaunay`. (ii) For each point, circumnavigate it. (We saw how to do this in our implementation of `Voronoi`, above.) (iii) Pick the shortest of the edges with the point at one end.

Convex Hull. Sometimes you may need to know the convex hull of a set of points in the plane for some other application. Although it might seem wasteful to construct the whole Delaunay triangulation just to get the hull, doing so is not too bad a method. Better efficiency can be achieved by ignoring, during the triangulation, points that are found to be already inside interior triangles. To sort the edges into the order of a CCW polygon, we create a `nextpt` table of edge destinations as we go, then chain through it once to output the vertices of the convex hull in proper order.

delaunay.h

```
struct Convexhull : Delaunay {
```
Structure for constructing the convex hull of a set of points in the plane. After construction, `nhull` is the number of points in the hull, and `hullpoints[0..nhull-1]` are integers pointing to points in the vector `pvec` that are in the hull, in CCW order.

```
    Int nhull;
    Int *hullpts;
    Convexhull(vector< Point<2> > pvec);          Construct from a vector of points.
};

Convexhull::Convexhull(vector< Point<2> > pvec) : Delaunay(pvec,2), nhull(0) {
```
Constructor for convex hull of a vector of points `pvec`. Bit "2" sent to the `Delaunay` constructor tells it to ignore interior points when it can, for extra speed.

```
    Int i,j,k,pstart;
    vector<Int> nextpt(npts);
    for (j=0; j<ntree; j++) {                     Triangles with stat= −1 may contain
        if (thelist[j].stat != -1) continue;          hull segments.
        for (i=0,k=1; i<3; i++,k++) {             Need two valid points to qualify.
            if (k == 3) k=0;
            if (thelist[j].p[i] < npts && thelist[j].p[k] < npts) break;
        }
        if (i==3) continue;                       Case where failed to qualify.
        ++nhull;                          Yes! Put its other end in the lookup table, and save
        nextpt[(pstart = thelist[j].p[k])] = thelist[j].p[i];   its value in case it's
    }                                                               the last one we find.
    if (nhull == 0) throw("no hull segments found");
    hullpts = new Int[nhull];                     Now we know how many, can allocate.
    j=0;                                          One chain through the lookup table, start-
    i = hullpts[j++] = pstart;                        ing with pstart, gives the answer.
    while ((i=nextpt[i]) != pstart) hullpts[j++] = i;
}
```

Largest Empty Circle Problem. The largest empty circle whose center lies (strictly) inside the convex hull of a set of points in the plane has its center on a Voronoi vertex. So, you can find it by looping through the Voronoi vertices, calculating the radius of the largest circle centered on each one, and taking the maximum of these. Even better, loop through the Delaunay triangles, calculate the circumcenter of each, and pick the one with the largest circumradius (since Delaunay circumcenters are exactly Voronoi vertices). Think of yourself as finding the best location for a fast-food restaurant within the (convex) city limits, one that best avoids all the other fast-food restaurants.

Avoiding Obstacles. If you want to navigate around the plane staying as far as possible from a set of points, your path will be along the edges of a Voronoi diagram. Think of yourself as a fighter pilot avoiding enemy radars.

Minimum Spanning Tree. The *minimum spanning tree* (sometimes *Euclidean minimum spanning tree*) is the set of line segments of shortest total length that connect N points (see, e.g., Figure 21.7.4). Think of it as the highway map of the cheapest highway system that lets you visit all N cities. It is topologically a tree (i.e., has no loops) because if it did have a loop, you could save highway money by deleting one of the loop's segments.

The important theorem is: The minimum spanning tree is a subset of the Delaunay edges. You might think this isn't very useful, since it doesn't tell you *which* subset. Fortunately, there is a fast algorithm, *Kruskal's algorithm*, for doing the construction. The basic idea is to sort all the Delaunay edges by length, and then add them one at a time to the growing tree, in order from smallest to largest.

Your tree starts off growing in multiple disconnected components, but when

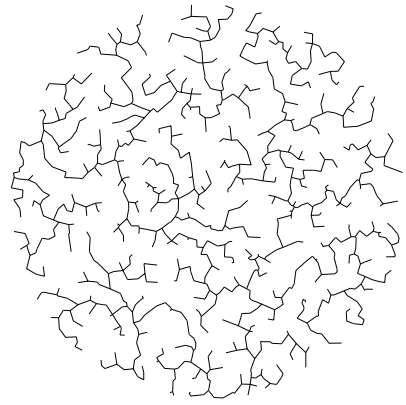


Figure 21.7.4. Minimum spanning tree of 1001 points at random locations within a circle. The tree is composed of 1000 segments that connect all the points with the shortest total length and is a subset of the Delaunay triangulation of the same points.

you have added exactly $N - 1$ segments, it will be a single piece, and the answer. There is only one catch: As you add segments, you must not add a segment if both its ends are already in the same component (else it would form a loop). So, you have to maintain an "equivalence class" relation for each vertex, making it equivalent to all other vertices in its connected component. We already know how to do this efficiently, as in §8.6's routine `eclass`. In the code below, there is a similar logic of sweeping up pointers to single "mother" representatives. Do this properly, and the method is $O(N \log N)$.

Kruskal's algorithm is a so-called *greedy algorithm*, since it just takes the best edge at each step willy nilly. It is rare for a greedy algorithm to yield the true global optimum; but this is the happy case where it does.

delaunay.h

```
struct Minspantree : Delaunay {
```

Structure for constructing the minimum spanning tree of a set of points in the plane. After construction, `nspan` is the number of segments (always = `npts`−1), and `minsega[0..nspan-1]` and `minsegb[0..nspan-1]` contain integers pointing to points in the vector `pvec` that are the two ends of each segment.

```
    Int nspan;
    VecInt minsega, minsegb;            Allocate arrays for the output.
    Minspantree(vector< Point<2> > pvec);
};

Minspantree::Minspantree(vector< Point<2> > pvec) :
    Delaunay(pvec,0), nspan(npts-1), minsega(nspan), minsegb(nspan) {
```

Constructor for the minimum spanning tree of a vector of points `pvec`. The `Delaunay` constructor gives the triangulation. We need only find the correct subset of edges.

```
    Int i,j,k,jj,kk,m,tmp,nline,n = 0;
    Triel tt;
    nline = ntri + npts -1;             Number of edges in the triangulation.
    VecInt sega(nline);                 Allocate working space for two ends of each edge,
    VecInt segb(nline);                     edge length, and index on which we will sort.
    VecDoub segd(nline);                    Also the "mother" tree for equivalence classes.
    VecInt mo(npts);
    for (j=0; j<ntree; j++) {           Find all edges in the triangulation, store them
        if (thelist[j].stat == 0) continue;     and their lengths.
        tt = thelist[j];
        for (i=0,k=1; i<3; i++,k++) {
            if (k==3) k=0;
```

```
            if (tt.p[i] > tt.p[k]) continue;   Ensure we get each edge only once.
            if (tt.p[i] >= npts || tt.p[k] >= npts) continue;   No edges connecting to fictitious points.
            sega[n] = tt.p[i];
            segb[n] = tt.p[k];
            segd[n] = dist(pts[sega[n]],pts[segb[n]]);
            n++;
        }
    }
    Indexx idx(segd);                  Sort the edges by creating an index array.
    for (j=0; j<npts; j++) mo[j] = j;  Initialize equivalence relation tree.
    n = -1;
    for (i=0; i<nspan; i++) {          Add exactly nspan segments.
        for (;;) {                     Loop for the shortest valid segment n.
            jj = j = idx.el(sega,++n);
            kk = k = idx.el(segb,n);
            while (mo[jj] != jj) jj = mo[jj];      Track each end to its highest ancestor.
            while (mo[kk] != kk) kk = mo[kk];
            if (jj != kk) {            The segment is valid only if it connects different highest ancestors.
                minsega[i] = j;
                minsegb[i] = k;
                m = mo[jj] = kk;       Now, equate the highest ancestors, and retrace our steps pointing all nodes encountered to that highest node, necessary for speed of the algorithm.
                jj = j;
                while (mo[jj] != m) {
                    tmp = mo[jj];
                    mo[jj] = m;
                    jj = tmp;
                }
                kk = k;
                while (mo[kk] != m) {
                    tmp = mo[kk];
                    mo[kk] = m;
                    kk = tmp;
                }
                break;                 A segment has been successfully added.
            }
        }
    }
}
```


CITED REFERENCES AND FURTHER READING:

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapters 7 and 11.

O'Rourke, J. 1998, *Computational Geometry in C*, 2nd ed. (Cambridge, UK: Cambridge University Press), Chapter 5.

21.8 Quadtrees and Octrees: Storing Geometrical Objects

Different from a KD tree is another kind of box tree, usually called a *quadtree* in two dimensions or a *octree* in three dimensions. Yes, we know that it ought to be spelled "octtree," not "octree," but the latter usage has become standard. We'll refer to quadtrees and octrees generically as "QO trees" and thus avoid linguistic controversy.

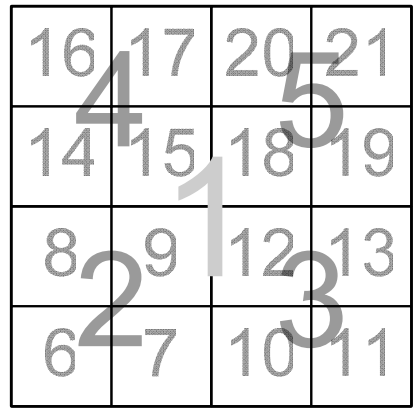


Figure 21.8.1. In a quadtree, the initial square 1 is first subdivided into squares 2, 3, 4, and 5. At the next level of subdivision, 2 is subdivided into 6, 7, 8, 9; 3 into 10, 11, 12, 13; and so forth.

QO trees start with a finite-sized box, usually square or cubical, rather than the near-infinite box used in the KD tree. A QO tree then subdivides each box not in one dimension at a time (as a KD tree) but rather in *all* dimensions. Thus, a square is subdivided into four daughter squares, a cube into eight daughter cubes — quite a brood. The coordinates of the subdivisions are taken to exactly bisect the mother box in each dimension, so all the boxes at one level of the tree are congruent, differing from the original box by a fixed power-of-two factor. Figure 21.8.1 illustrates this in the case of two dimensions.

QO trees thus provide a kind of addressing scheme for two- or three-dimensional space. Accordingly, they can be used to store and retrieve finite-sized geometrical objects that fit into the boxes of the tree at one or another level, and to test for intersections of such objects, for nearness relationships, and so on. The general idea (although there can be variations on this) is to store each object in the smallest box that completely contains it — or, in the case of a zero-sized object like a point, in the appropriate box at the deepest level of tree that we care to implement. Then, when doing a collision or nearness test, we traverse only those parts of the tree that are relevant, much as we did in the applications of KD trees.

Although we will illustrate only the most elementary of applications, QO trees are often at the heart of more complicated algorithms, for example [1-3],

- Hidden polygon removal in the visual plane (which projected polygons intersect a given pixel in the visual field?)
- Fast gravitational or Coulomb N-body calculations (store fictitious objects on various scales that sweep up the multipole moments of the collections of point masses that they contain) [4,5,6]
- Mesh generation (choose a local mesh scale to match the scale in the QO tree at which obstacles or boundaries are stored; the concept of a *balanced* QO tree is often used)
- Image compression (store slowly varying parts of the image as objects high in the tree, and prune unnecessary daughters).

A main weakness of QO trees follows just from their geometrical regularity. If a finite-sized object being stored in a QO tree falls on the boundary between two

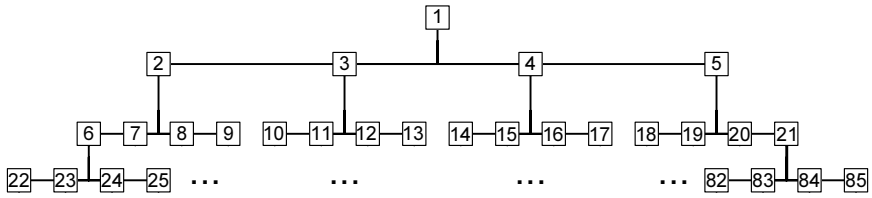


Figure 21.8.2. Quadtree shown in tree form. Because of its regularity, a quadtree's relationships can be described numerically. For example, the mother of box n is the integer part of $n/4$. The left daughter of box n is $4n - 2$.

boxes of about its size, then it can't be stored in either of them. Instead, it gets stored in the larger — sometimes very much larger — box that first completely contains it. If N "small" objects are stored, then the number that fall on boundaries of the highest-level boxes scales in two dimensions as $N^{1/2}$, or in three dimensions as $N^{2/3}$. These objects will thus end up stored in just a few boxes at the top of the tree, and they will participate in almost all operations that check for collisions or nearness. Thus, QO trees can usually effect a time savings that turns a naive algorithm scaling as N into one that scales as $N^{1/2}$ (in the two-dimensional case); but only rarely, or with specialized methods, can they get to that Nirvana of $\log N$ or constant scaling. Still, the square- (or, in three dimensions, cube-) root of a large number can be a large factor, and well worth saving in time. So QO trees are good to know about.

That same geometrical regularity of QO trees allows them to be implemented, at least optionally, as an efficient hash structure where most boxes in the tree, if they are empty, require no storage space. We will give such an implementation here, both for its intrinsic advantages and because it is fairly concise to code, using the `Hash` and `Mhash` classes from §7.6.

The key observations follow from Figure 21.8.2, which shows a QO tree laid out in tree form. The boxes are numbered as in Figure 21.8.1, starting with box 1 at the root of the tree. With this numbering scheme, there exist simple numerical relations between the mother and daughter boxes. This enables one to navigate the tree — up, down, and sideways — without the use of any stored pointers. In particular, if $k \geq 1$ is a box's number, the following relationships hold in D (for us, two or three) dimensions:

$$\begin{aligned} \text{mother } (k) &= \lfloor (k + 2^D - 2)/2^D \rfloor \\ \text{leftmost daughter } (k) &= 2^D k - 2^D + 2 \\ \text{rightmost daughter } (k) &= 2^D k + 1 \\ \text{total boxes through level p} &= [(2^D)^p - 1]/(2^D - 1) \end{aligned} \tag{21.8.1}$$

Note that the integer divide by 2^D implied by the $\lfloor\ \rfloor$ notation can be implemented simply as a right shift by D bits. You should check the formulas in (21.8.1) against Figure 21.8.2 to be sure that you understand how they work. The "levels" of the tree are numbered starting with $p = 1$ for (only) box 1. Notice that the mother of box 1 computes to 0, indicating that it has no mother; this is convenient for testing when to exit loops over ancestry.

Before we get to implementation details for the implementing class `Qotree`, we need to discuss the prerequisites for a class of geometrical objects to be stored

in the tree. `Qotree` will be templated by a type parameter `elT` representing those objects. To store an object `myel` of type `elT`, you must be sure to provide a method `myel.isinbox()` whose argument is a `Box`, and which returns 1 if `myel` is in the `Box`, or 0 otherwise. Similarly, to erase an object, you need to provide an `==` operator, to decide (by comparison) which is the object to be erased. Those two methods are all that `Qotree` needs for itself. However, many applications of `Qotree` (including some that we illustrate later in this section) need either or both of the methods `myel.contains()` and `myel.collides()`, the first returning whether `myel` contains a given `point`, the second returning whether `myel` collides with another element of type `elT`.

Here is a simple example of a class, representing a circle (when `DIM` is 2) or sphere (when `DIM` is 3), that has these methods and can thus be stored and processed with a `Qotree`:

sphcirc.h

```
template<Int DIM> struct Sphcirc {
Circle (DIM=2) or sphere (DIM=3) object, with methods suitable for use with Qotree.
    Point<DIM> center;
    Doub radius;
    Sphcirc() {}                          Default constructor is needed to make arrays.
    Sphcirc(const Point<DIM> &mycenter, Doub myradius)   Construct by explicit center
        : center(mycenter), radius(myradius) {}              and radius.
    bool operator== (const Sphcirc &s) const {           Test if identical.
        return (radius == s.radius && center == s.center);
    }
    Int isinbox(const Box<DIM> &box) {          Is the circle/sphere inside a box?
        for (Int i=0; i<DIM; i++) {
            if ((center.x[i] - radius < box.lo.x[i]) ||
                (center.x[i] + radius > box.hi.x[i])) return 0;
        }
        return 1;
    }
    Int contains(const Point<DIM> &point) {   Is a given point inside the circle/sphere?
        if (dist(point,center) > radius) return 0;
        else return 1;
    }
    Int collides(const Sphcirc<DIM> &circ) {  Does it collide with another circle/sphere?
        if (dist(circ.center,center) > circ.radius+radius) return 0;
        else return 1;
    }
};
```

21.8.1 A Hashed QO Tree Implementation

We will implement the QO tree using two hash memories. First, there is an `Mhash` multimap memory (called `elhash`) whose keys are box numbers and whose stored elements are the geometrical objects that may be stored in the QO tree, with possibly multiple objects in a single box. Second, there is a single-valued `Hash` memory (called `pophash`) that associates an integer with every box that either (i) contains one or more elements (is "populated"), or (ii) is an ancestor to a box that is populated. In that integer, bit 0 (least significant bit) is used to indicate whether the box is populated, while bits $1 \ldots 2^D$ (that is, $1 \ldots 4$ or $1 \ldots 8$) are used to indicate which (if any) daughters are themselves either populated or are an ancestor to a populated box. In other words, `pophash`, combined with the relationships in equation (21.8.1), substitutes for the entire structure of doubly linked pointers that might more conventionally implement the tree.

The maximum number of levels $p_{\max}$ that we can represent is limited only by the largest value that can be represented by the integer type that stores box numbers k. Using 32-bit signed integers, 16 levels are possible in two dimensions, since $(4^{16} - 1)/3 < 2^{31} - 1$

(cf. equation 21.8.1). In three dimensions, 11 levels can be represented, since $(8^{11}-1)/7 < 2^{31}-1$. Often there is no need for this much resolution ($\sim 10^9$ boxes), so we will provide for setting a smaller value of $p_{\max}$, a good idea since the time to traverse one branch of the tree from root to leaf (a frequently occurring "atom" in other procedures) scales linearly with $p_{\max}$.

qotree.h

```
template<class elT, Int DIM> struct Qotree {
```

Quadtree (DIM=2) or octree (DIM=3) object to store geometrical objects of type elT.

```
    static const Int PMAX = 32/DIM;          Roughly how many levels fit in 32 bits.
    static const Int QO = (1 << DIM);        I.e., 4 for quad-, 8 for oct-.
    static const Int QL = (QO - 2);          Offset constant to leftmost daughter.
    Int maxd;
    Doub blo[DIM];
    Doub bscale[DIM];
    Mhash<Int,elT,Hashfn1> elhash;           Contains stored elements hashed by box #.
    Hash<Int,Int,Hashfn1> pophash;           Contains node population info.
    Qotree(Int nh, Int nv, Int maxdep);      The constructor. See below.
    void setouterbox(Point<DIM> lo, Point<DIM> hi);  Set scale and position.
    Box<DIM> qobox(Int k);                   Return the box whose number is k.
    Int qowhichbox(elT tobj);                Return smallest box containing tobj.
    Int qostore(elT tobj);                   Store an elT object in the Qotree.
    Int qoerase(elT tobj);                   Erase an elT object in the Qotree.
    Int qoget(Int k, elT *list, Int nmax);     Retrieve all objects in box k.
    Int qodump(Int *k, elT *list, Int nmax);   Retrieve all objects.
    Int qocontainspt(Point<DIM> pt, elT *list, Int nmax);   See below.
    Int qocollides(elT qt, elT *list, Int nmax);            See below.
};
```

```
template<class elT, Int DIM>
Qotree<elT,DIM>::Qotree(Int nh, Int nv, Int maxdep) :
    elhash(nh, nv), maxd(maxdep), pophash(maxd*nh, maxd*nv) {
```

Constructor for a quad- (DIM=2) or oc- (DIM=3) tree that can store a max of nv elements of type elT, using hash tables of length nh (typically $\approx$ nv). maxdep is the number of levels to be represented.

```
    if (maxd > PMAX) throw("maxdep too large in Qotree");
    setouterbox(Point<DIM>(0.0,0.0,0.0),Point<DIM>(1.0,1.0,1.0));  Default scale.
}
```

```
template<class elT, Int DIM>
void Qotree<elT,DIM>::setouterbox(Point<DIM> lo, Point<DIM> hi) {
```

Sets the scale of Qotree to an outer box defined by points lo and hi. Must be called before any elements are stored in the tree.

```
    for (Int j=0; j<DIM; j++) {
        blo[j] = lo.x[j];
        bscale[j] = hi.x[j] - lo.x[j];
    }
}
```

You will normally call setouterbox() immediately after invoking the qotree constructor to create a QO tree. Otherwise, you get the default box with a corner at the origin and unit size in each dimension.

Right away, we need two utility routines. The first takes a box's number (e.g., as in Figure 21.8.1) and returns the actual box (as a Box<DIM>). The second takes an object of the type to be stored in the tree (elT) and returns the number of the smallest box that contains it. It does this by starting at the top of the tree, trying each possible daughter, and moving deeper into the tree only when a containment is found.

qotree.h

```
template<class elT, Int DIM>
Box<DIM> Qotree<elT,DIM>::qobox(Int k) {
```

Returns the box indexed by k.

```
    Int j, kb;
    Point<DIM> plo, phi;
```

```
    Doub offset[DIM];
    Doub del = 1.0;
    for (j=0; j<DIM; j++) offset[j] = 0.0;
    while (k > 1) {                          Up through ancestors until get to root.
        kb = (k + QL) % QO;                  Which daughter is k? Add its offset.
        for (j=0; j<DIM; j++) { if (kb & (1 << j)) offset[j] += del; }
        k = (k + QL) >> DIM;                 Replace k by its mother,
        del *= 2.0;                          where offsets will be twice as big.
    }
    for (j=0; j<DIM; j++) {                  At the end, scale the offsets by the final del to
        plo.x[j] = blo[j] + bscale[j]*offset[j]/del; make them metrically correct.
        phi.x[j] = blo[j] + bscale[j]*(offset[j]+1.0)/del;
    }
    return Box<DIM>(plo,phi);                Construct the box and return it.
}

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qowhichbox(elT tobj) {
```

Return the box number of the smallest box that can contain an element `tobj`, without regard to whether `tobj` is already stored in the tree.

```
    Int p,k,kl,kr,ks=1;                      Answer is box 1 unless a smaller box found.
    for (p=2; p<=maxd; p++) {                Go down through the levels.
        kl = QO * ks - QL;                   Leftmost daughter.
        kr = kl + QO -1;                     Rightmost daughter.
        for (k=kl; k<=kr; k++) {             Do any daughters contain tobj?
            if (tobj.isinbox(qobox(k))) { ks = k; break; }
        }
        if (k > kr) break;                   No. Therefore, discontinue descent here.
    }
    return ks;
}
```

Now we are ready to store elements into the tree, or to erase elements previously stored. With `qowhichbox()`, above, and the methods that belong to the `Mhash`, it is trivial to do the actual store or erase. Trickier to code is to create or erase the trail of "breadcrumbs" in `pophash` that connect the box to its ancestors. When we erase, we must be sure not to cut off the trail to any remaining elements in the same box, or to elements in descendant boxes.

qotree.h

```
template<class elT, Int DIM>
Int Qotree<elT,DIM>::qostore(elT tobj){
```

Store the element `tobj` in the `Qotree`, and return the box number into which it was stored.

```
    Int k,ks,kks,km;
    ks = kks = qowhichbox(tobj);
    elhash.store(ks, tobj);                  Store the element in elhash
    pophash[ks] |= 1;                        and mark its box as populated.
    while (ks > 1){                          Now leave trail of breadcrumbs to the root mother.
        km = (ks + QL) >> DIM;               Mother of ks.
        k = ks - (QO*km - QL);               Which daughter of km is ks.
        ks = km;                             Now set the daughter bit in the mother.
        pophash[ks] |= (1 << (k+1));
    }
    return kks;
}

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qoerase(elT tobj) {
```

Erase the element `tobj`, returning the box number into which it was stored or 0 if the element was not found in the `Qotree`. Note logic very similar to `qostore`.

```
    Int k,ks,kks,km;
    Int *ppop;
    ks = kks = qowhichbox(tobj);             Find the box.
    if (elhash.erase(ks, tobj) == 0) return 0;   It ain't there!
    if (elhash.count(ks)) return kks;        Sisters still in same box, so we are done.
```

```
    ppop = &pophash[ks];                    Must now erase any unneeded breadcrumbs.
    *ppop &= ~((Uint)1);                    Unmark the pop bit.
    while (ks > 1) {                        Up through the ancestors...
        if (*ppop) break;                   Box is populated or has daughters, so done.
        pophash.erase(ks);                  Erase unneeded (zero) pophash entry.
        km = (ks + QL) >> DIM;              Mother of ks.
        k = ks - (QQ*km - QL);              Which daughter of km is ks.
        ks = km;
        ppop = &pophash[ks];
        *ppop &= ~((Uint)(1 << (k+1)));     Unset the daughter bit in the mother.
    }
    return kks;
}
```

Finally, we need methods to retrieve elements previously stored, either those in a given box (by number), or else all the elements in the tree. In the former case, the `Mhash` does all the work. In the latter case, however, we must provide the machinery for a recursive search of the tree, since at any stage we may encounter a box with multiple populated daughters. Notice that the calling routine is responsible for supplying storage (as an array `list[]`) for the result and declaring the maximum number `nmax` of elements that it is prepared to accept.

qotree.h

```
template<class elT, Int DIM>
Int Qotree<elT,DIM>::qoget(Int k, elT *list, Int nmax) {
```

Retrieve all (or up to `nmax` if it is smaller) elements that are stored in box `k` of the `Qotree`. The elements are copied into `list[0..nlist-1]` and the value `nlist` ($\leq$ `nmax`) is returned.

```
    Int ks, pop, nlist;
    ks = k;
    nlist = 0;
    pophash.get(ks,pop);
    if ((pop & 1) && elhash.getinit(ks)) {
        while (nlist < nmax && elhash.getnext(list[nlist])) {nlist++;}
    }
    return nlist;
}

template<class elT, Int DIM>
Int Qotree<elT,DIM>::qodump(Int *klist, elT *list, Int nmax) {
```

Retrieve all (or up to `nmax` if it is smaller) elements that are stored anywhere in the `Qotree`, along with their corresponding box numbers. The elements are copied into `list[0..nlist-1]` and the value `nlist` ($\leq$ `nmax`) is returned. The box numbers are copied into `klist[0..nlist-1]`.

```
    Int nlist, ntask, ks, pop, k;
    Int tasklist[200];                      Stack of pending box numbers as we recur-
    nlist = 0;                                  sively traverse the tree.
    ntask = 1;
    tasklist[1] = 1;
    while (ntask) {                         As long as tasks remain...
        ks = tasklist[ntask--];
        if (pophash.get(ks,pop) == 0) continue;   Box empty and no daughters.
        if ((pop & 1) && elhash.getinit(ks)) {    The box is populated, so we output
            while (nlist < nmax && elhash.getnext(list[nlist])) {  its contents.
                klist[nlist] = ks;
                nlist++;
            }
        }
        if (nlist == nmax) break;           No more room for output!
        k = QQ*ks - QL;                     Leftmost daughter.
        while (pop >>= 1) {                 Loop over the daughter bits in pop.
            if (pop & 1) tasklist[++ntask] = k;   Daughter exists. Add to task list.
            k++;                            Next daughter.
        }
    }
    return nlist;
}
```

The additional functions declared in Qotree pertain to applications, as we now discuss.

21.8.2 QO Tree Elementary Applications

Two important building blocks for applications of QO trees are, first, a routine that returns a list of all stored elT elements that intersect (i.e., contain) a specified point; and, second, a routine that returns a similar list of all stored elT elements that intersect (i.e., collide with) a specified elT element.

An element that intersects a point will evidently be stored in a box that is an ancestor to the box that the point is in, or else in the same box as the point. It takes just one pass down through the levels of the tree to find all such elements.

qotree.h

```
template<class elT, Int DIM>
Int Qotree<elT,DIM>::qocontainspt(Point<DIM>pt, elT *list, Int nmax) {
```

Retrieve all (or up to nmax if it is smaller) elements in Qotree that contain the point pt. The elements are copied into list[0..nlist-1] and the value nlist ($\leq$ nmax) is returned.

```
    Int j,k,ks,pop,nlist;
    Doub bblo[DIM], bbscale[DIM];
    for (j=0; j<DIM; j++) { bblo[j] = blo[j]; bbscale[j] = bscale[j]; }
    nlist = 0;
    ks = 1;                                  Start at the top of the tree.
    while (pophash.get(ks,pop)) {            Descend as long as something is there.
        if (pop & 1) {                       The box is populated, so we check its con-
            elhash.getinit(ks);                  tained elements,
            while (nlist < nmax && elhash.getnext(list[nlist])) {
                if (list[nlist].contains(pt)) {nlist++;} returning any that contain
            }                                                    pt.
        }
        if ((pop >>= 1) == 0) break;         The box has no daughters, so we are done.
        for (k=0, j=0; j<DIM; j++) {         Compute k, the single daughter containing
            bbscale[j] *= 0.5;                   pt.
            if (pt.x[j] > bblo[j] + bbscale[j]) {
                k += (1 << j);
                bblo[j] += bbscale[j];
            }
        }
        if (((pop >> k) & 1) == 0) break;    No such daughter exists in the tree.
        ks = QO * ks - QL + k;               Daughter exists and is the next node to check.
    }
    return nlist;
}
```

When an element A intersects another element B, either A and B are in the same box, or else A is in an ancestor box to B, or else B is in an ancestor box to A. Equivalently, for a fixed A, we can find all intersecting B's by searching A's box, its ancestors, and its descendants. The latter search requires a task list stack, as we have seen before (e.g., in qodump).

qotree.h

```
template<class elT, Int DIM>
Int Qotree<elT,DIM>::qocollides(elT qt, elT *list, Int nmax) {
```

Retrieve all (or up to nmax if it is smaller) elements in Qotree that collide with an element qt (which needn't be in the tree itself). The elements are copied into list[0..nlist-1] and the value nlist ($\leq$ nmax) is returned.

```
    Int k,ks,kks,pop,nlist,ntask;
    Int tasklist[200];                   Stack of pending box numbers.
    nlist = 0;
    kks = ks = qowhichbox(qt);           kks saves the starting box.
    ntask = 0;
    while (ks > 0) {                     Put the starting box and all its ancestors on the
                                             task list.
```

```
            tasklist[++ntask] = ks;
            ks = (ks + QL) >> DIM;          Move to mother.
        }
        while (ntask) {
            ks = tasklist[ntask--];
            if (pophash.get(ks,pop) == 0) continue;    Box empty and no daughters.
            if (pop & 1) {                  The box is populated, so we check its contained
                elhash.getinit(ks);             elements,
                while (nlist < nmax && elhash.getnext(list[nlist])) {
                    if (list[nlist].collides(qt)) {nlist++;}     returning any that col-
                }                                                    lide with qt.
            }
            if (ks >= kks) {                Recurse only for descendants, not ancestors!
                k = QO*ks - QL;             Leftmost daughter.
                while (pop >>= 1) {
                    if (pop & 1)            Daughter exists. Add to task list.
                        tasklist[++ntask] = k;
                    k++;                    Next daughter.
                }
            }
        }
        return nlist;
    }
```

As an example of a simple application of a QO tree, let's replicate the functionality of `KDtree::locatenear` (§21.2) with a routine that finds all stored points within a specified radius r of a test point. Using the class `Sphcirc`, points are represented as circles/spheres of zero radius, the test point as a circle/sphere of radius r, and we use `qocollides` to find the collisions.

We implement this application as a structure, `Nearpoints`, whose constructor creates the QO tree out of a vector of points, and whose member function `locatenear` can then be called to find all stored points within any specified radius of any specified point.

qotree.h

```
template <int DIM> struct Nearpoints {
```

Object for constructing a QO tree containing a set of points, and for repeatedly querying which stored points are within a specified radius of a specified new point.

```
    Int npts;
    Qotree<Sphcirc<DIM>,DIM> thetree;
    Sphcirc<DIM> *sphlist;
    Nearpoints(const vector< Point<DIM> > &pvec)
        : npts(pvec.size()), thetree(npts,npts,32/DIM) {
```

Constructor. Creates the QO tree from a vector of points pvec.

```
        Int j,k;
        sphlist = new Sphcirc<DIM>[npts];
        Point<DIM> lo = pvec[0], hi = pvec[0];    Find bounding box for the points.
        for (j=1; j<npts; j++) for (k=0; k<DIM; k++) {
            if (pvec[j].x[k] < lo.x[k]) lo.x[k] = pvec[j].x[k];
            if (pvec[j].x[k] > hi.x[k]) hi.x[k] = pvec[j].x[k];
        }
        for (k=0; k<DIM; k++) {                   Expand it by 10% so that all points
            lo.x[k] -= 0.1*(hi.x[k]-lo.x[k]);         are well interior.
            hi.x[k] += 0.1*(hi.x[k]-lo.x[k]);
        }
        thetree.setouterbox(lo,hi);     Set the tree's outer box and store all the points.
        for (j=0; j<npts; j++) thetree.qostore(Sphcirc<DIM>(pvec[j],0.0));
    }
    ~Nearpoints() { delete [] sphlist; }
    Int locatenear(Point<DIM> pt, Doub r, Point<DIM> *list, Int nmax) {
```

Once the tree is constructed, this function can be called repeatedly with varying points `pt` and radii `r`. It returns `n`, the number of stored points within radius `r` of `pt` (but no larger than `nmax`), and copies those points into `list[0..n-1]`.

```
        Int j,n;
        n = thetree.qocollides(Sphcirc<DIM>(pt,r),sphlist,nmax);
        for (j=0; j<n; j++) list[j] = sphlist[j].center;
        return n;
    }
};
```

In practice, the routine above is rather slower than `KDtree::locatenear` for this application, because there is a lot of overhead involved in copying `Point` and `Sphcirc` elements around, and in computing `Boxes` as we drill down the tree. By contrast, `KDtree` is lean and mean, since it only stores points and, in our implementation, copies them internally to a fast store of coordinates.

Unlike the KD tree, however, the technique illustrated here can be generalized to much more complicated situations. For example, instead of being simple points, the stored objects could be broadcast reception areas for FM radio stations on a given frequency, and we might want to know where collisions occur with proposed new stations. The `collides()` function between two broadcast areas might involve a lengthy calculation taking into account their powers, the surrounding detailed topography, and so on. In such a case, the overhead of the QO tree might well be negligible as we seek to minimize the number of calls to `collides()`.

As a second example of a simple application, consider a square Petri dish on which spores land, in random positions, one at a time. Each such spore quickly grows into a circular colony that just touches the nearest existing colony (or the edge of the dish), and then stops. (Don't ask us why. This is only an example.) What does the dish look like after N spores have landed?

Rather than give the code in detail, a simple description should suffice: The objects stored in the QO tree are circles. Looping over the number of spores, we pick a random location for each in turn. If the QO tree method `qocontainspt()` indicates that the location lies within an already-stored colony, go on to the next spore. Otherwise, start with a small trial radius and increase it (by doubling, e.g.) until `qotreecollides()` first indicates collisions. Now adjust the trial radius to be the minimum of distances to the colliding elements, add that colony to the tree, and go on to the next spore.

Figure 21.8.3 shows an example of the resulting configuration, after 1000 colonies have grown. (Another 3592 spores landed inside existing colonies and died immediately.)

CITED REFERENCES AND FURTHER READING:

de Berg, M., van Kreveld, M., Overmars, M., and Schwarzkopf, O. 2000, *Computational Geometry: Algorithms and Applications*, 2nd revised ed. (Berlin: Springer), Chapter 14.[1]

Samet, H. 1990, *The Design and Analysis of Spatial Data Structures* (Reading, MA: Addison-Wesley).[2]

Samet, H. 1990, *Applications of Spatial Data Structures: Computer Graphics, Image Processing, and GIS* (Reading, MA: Addison-Wesley).[3]

Pfalzner, S. and Gibbon, P. 1996, *Many-Body Tree Methods in Physics* (Cambridge, UK: Cambridge University Press).[4]

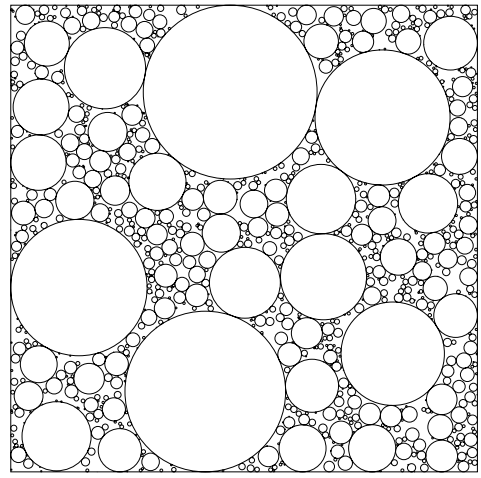


Figure 21.8.3. Spores land randomly on a square (!) Petri dish, and grow to colonies that barely touch the nearest pre-existing colony, or the edge of the dish. A QO tree can be used to keep track of collisions. Here, 1000 colonies have grown to their maximum size.

Greengard, L., and Wandzura, S., eds. 1998, "Fast Multipole Methods," special issue of *IEEE Computational Science and Engineering*, vol. 5, no. 3 (July–September), pp. 16–56.[5]

Gumerov, N.A., and Duraiswami, R. 2004, *Fast Multipole Methods for the Helmholtz Equation in Three Dimensions* (Amsterdam: Elsevier).[6]